A practical engineering architecture for long-lived cognitive systems, continuous memory consolidation, objective-driven world models, communication, hybrid computation, persistent memory, and controlled architectural evolution.

Chapters

10. Chapter 10: Associative Memory

    Defines the canonical memory interface and the dynamic associative-memory reference architecture introduced in cognitive-0.3.17.

11. Chapter 11: Canonical Memory Representation

    Defines the Memory Vector contract, semantic stability, localized basis maintenance, effective dimensionality, and dense progressive semantic compaction.

12. Chapter 12: Learning and Memory Update

    Defines online learning, controlled memory updates, staged integration of the Transformer and Associative Memory, maturity criteria, persistence, rollback, and consolidation triggers.

13. Chapter 13: Sleep and Structural Learning

    Defines future offline structural optimization, topology adaptation, connection-state evidence, and the boundary between functional and structural learning.

14. Chapter 14: Hybrid Neural and Algorithmic Computation

    Allocates operations among neural, algorithmic, finite-state, and hybrid mechanisms.

15. Chapter 15: Cognitive Transformation through Language

    Describes documents, lectures, dialogue, and education as controlled transformations of a cognitive model.

16. Chapter 16: Population, Deployment, and Governance

    Addresses cognitive lineages, diversity, experience exchange, deployment, risk, and governance.

17. Chapter 17: Specification Model and Tooling

    Defines the canonical YAML model, atomic identifiers, validation, generated documentation, Git workflow, and self-hosting boundaries.

18. Chapter 18: Implementation Roadmap and Open Questions

    Separates implemented infrastructure, near-term experiments, deferred mechanisms, unresolved questions, and prospective dense photonic-spintronic hardware realization.

19. Chapter 20: Architectural Evolution Roadmap

    Defines the ordered evolution from semantic learning toward importance estimation, sleep, replay, connection-state evidence, structural plasticity, evaluation, and multi-agent learning.

20. Chapter 21: Research Agenda

    Translates future architectural directions into falsifiable research programs, experiments, measurements, and acceptance criteria.

21. Chapter 22: Long-Term Vision

Describes the intended evolution from semantic reasoning with persistent memory toward a continuously improving cognitive platform.

22. Chapter 23: Open Research Questions

    Records unresolved questions that must remain explicit until evidence supports architectural decisions.

23. Chapter 24: Documentation Architecture

    Defines how canonical project knowledge is typed, identified, related, generated, validated, and evolved under Single Source of Truth.

24. Chapter 25: Typed Knowledge Appendices

    Provides generated registries for Research Notes, Architectural Notes, Implementation Notes, and Historical Perspectives.

25. Chapter 26: Alphabetical Index

    Alphabetical access to chapters, sections, typed knowledge objects, and canonical terms.

Project References

- Token Registry
- Canonical YAML Model
- Documentation Style Guide

1.1 Purpose

This document develops an engineering architecture for long-lived cognitive systems capable of accumulating experience, reorganizing knowledge, preserving continuity, and improving throughout an operational lifetime measured in years or decades.

The immediate objective is not to claim a finished general intelligence architecture. The objective is to identify the next implementable architectural changes that provide the greatest increase in practical utility for foreseeable engineering cost while increasing the number of useful future development paths.

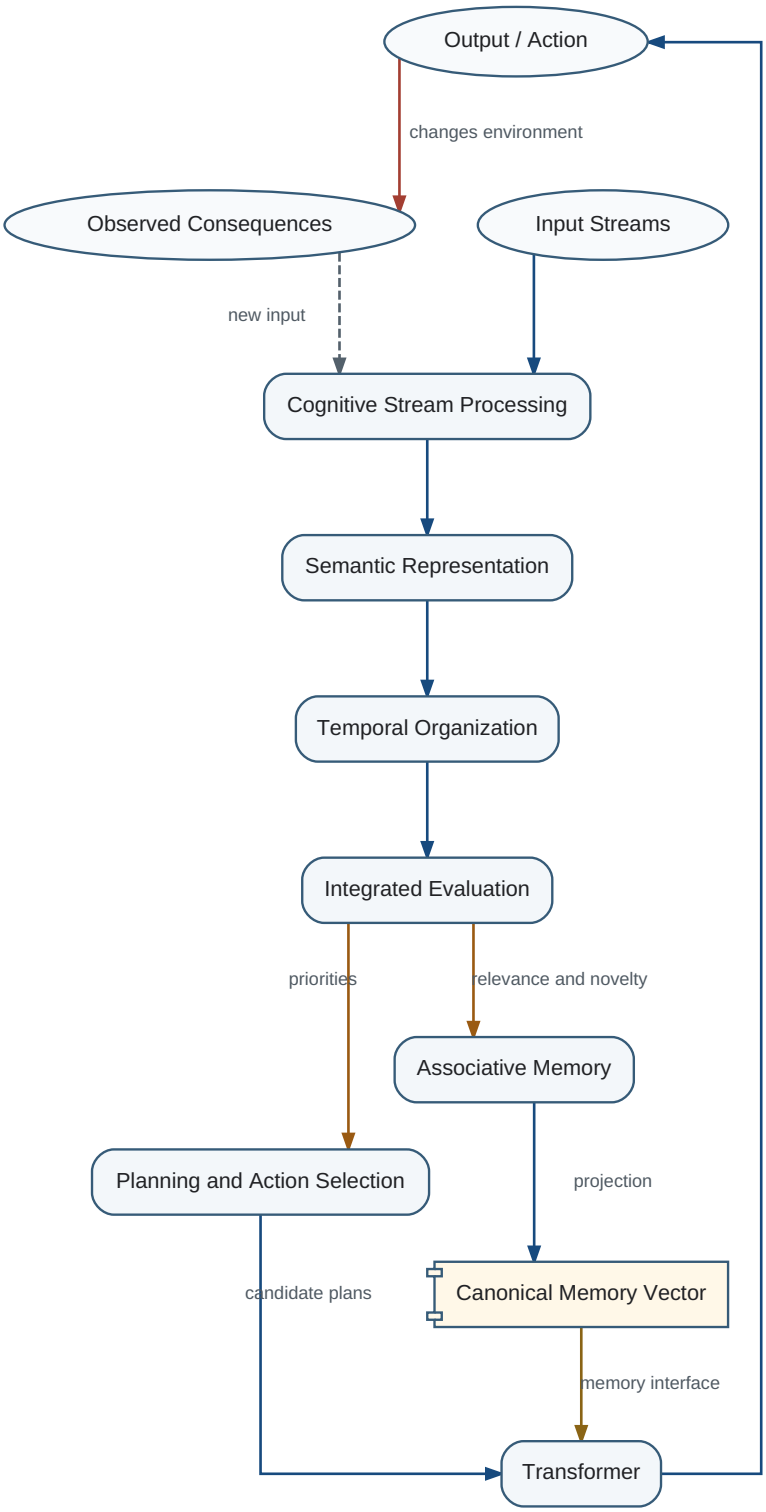
> **Primary optimization criterion**
>
> Select the next architectural step by the ratio of practical utility to implementation cost, adjusted for risk and for the additional future options enabled by that step.

The project therefore treats existing neural models, classical programs, storage systems, version-control systems, and document generators as reusable components. Novel work is concentrated on their organization, interfaces, persistent state, and long-term evolution.

1.2 Architecture Before Component Complexity

Many improvements in current systems are obtained by increasing model scale or adding specialized components. This project instead examines whether a different organization of existing components can create a larger gain than making any one component more complex.

Overall Cognitive Architecture



The end-to-end relationship between input streams, semantic processing, evaluation, planning, memory, reasoning, action, and feedback.

The strategy is evolutionary. Each stage must be fully operational, independently useful, measurable, and capable of supplying evidence for the next stage. Tighter integration is deferred until the modular implementation has generated enough operational data to justify it.

```
Existing Components ↓
Improved Organization ↓
Operational Evidence ↓
Deeper Integration
```

> **No speculative dependency**
>
> A proposed mechanism belongs in the engineering roadmap only when it can be implemented with current technology, requires an identifiable extension of current technology, or can be experimentally tested in the foreseeable future.

1.3 Scope

The project models computational and architectural mechanisms: memory, knowledge organization, evaluation, communication, continuous learning, architectural search, identity continuity, and allocation of work between neural and algorithmic computation.

Social, psychological, emotional, ethical, and political phenomena are not independently engineered in the current stage. The working hypothesis is that a substantial subset of these phenomena will emerge from sufficiently mature cognitive architecture. This is a hypothesis, not an established fact.

> **Boundary of the claim**
>
> The project does not assume that every higher-level behavior must emerge. Persistent failures to emerge are evidence for adding explicit mechanisms at a later stage.

Subjective consciousness is not required by the functional architecture described here. A system may possess continuity, goals, and a persistent self-model without any proven subjective experience.

1.4 Epistemic Status

The document separates implemented mechanisms, architectural decisions, working hypotheses, observations, and open questions. Confidence values are engineering estimates, not statistical posterior probabilities unless an experiment explicitly establishes that interpretation.

Biological evolution is treated as a very large empirical search that produced robust cognitive architectures under severe real-world constraints. Evolutionary persistence makes a mechanism a strong candidate for study, but does not prove that the mechanism is optimal for an engineered system with different resources and objectives.

A good project model must remain corrigible. New evidence may refine definitions, change confidence, replace architecture decisions, or create new Tokens. Existing Tokens do not change semantic identity; semantic replacement requires a new Token.

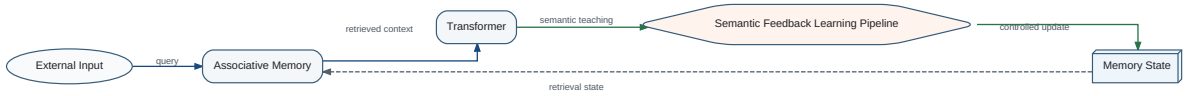
> ### State recovery
>
> The canonical Research State and generated documentation must permit a new human or model participant to resume the project after context loss without replaying the complete discussion history.

Cognitive separates semantic reasoning, associative retrieval, persistent memory state, and self-learning while connecting them through explicit typed interfaces.

Primary architectural principle

The Transformer is both the Semantic Reasoning Engine and the Semantic Teacher of Associative Memory. Associative Memory learns from Transformer-produced Semantic Representations rather than raw external observations.

Cognitive Architecture — Functional Overview



A compact system-level view. The detailed closed learning lifecycle and operation contracts are defined canonically in Section 10.1.

Canonical detailed specification

Section 10.1 owns the complete Semantic Feedback Learning lifecycle, the READ and UPDATE contracts, and the rule that UPDATE never produces a Memory Vector. This overview intentionally does not repeat that complete definition.

Operation asymmetry

Operation	Architectural purpose	Canonical owner
READ	Retrieve context without changing memory	Section 10.1
UPDATE	Incorporate semantic learning into Memory State	Section 10.1

The release-blocking READ/UPDATE invariants are defined canonically in Section 10.1 and in state/canonical/invariants.yaml.

First-class self-learning process

The Semantic Feedback Learning Pipeline is an independent architectural object with explicit stages, contracts, invariants, and future extension points for consolidation, replay, forgetting, structural plasticity, and evaluation.

A finite cognitive system cannot represent every property of external reality. It must select distinctions, relations, and temporal scales. The selection is governed by what the system must predict, evaluate, or do.

No world model is produced directly from reality. Observations are first transformed into an internal knowledge model. That knowledge model is then weighted and organized relative to an objective function.

```
WorldModel = Optimize(KnowledgeModel(ObservedReality), ObjectiveFunct
```

The phrase universal world model is therefore misleading when applied to a finite system. A model can be broad and transferable, but it remains an optimization for a class of objectives and resource constraints.

> `T_ObjectiveFunction`
>
> A mechanism that evaluates candidate states relative to the objectives of a cognitive system.

An objective function influences far more than final action selection. Over time it changes what is attended to, what is stored, what is forgotten, which categories are created, which errors matter, and which communication is considered useful.

The objective function need not be explicit, scalar, stable, or fully consistent. In biological systems it is likely implemented by several interacting evaluation mechanisms. An engineered system can begin with an explicit approximation and allow the evaluation mechanism to learn.

System class	High-value distinctions	Low-value distinctions
Predator	Prey, motion, concealment, energy	Taxonomic detail unrelated to hunting
Herbivore	Food, predators, escape routes	Predator hunting strategy beyond avoidance
Scientist	Prediction, explanation, reproducibility	Immediate utility without explanatory value
Artist	Form, contrast, meaning, emotional effect	Mechanistic detail irrelevant to expression

A useful unit of design is not a unique world model but a bounded class of models associated with a bounded class of systems. Members of a class share objectives, representational conventions, and relevant environmental regularities while retaining individual histories.

This class-based view permits controlled cloning and commercial deployment without forcing every deployed system into one global cognitive state.

Models from different classes can remain mutually intelligible through explicit communication while preserving different internal organizations. Compatibility does not require identity.

> ### Class-limited optimality
>
> A model can be near-optimal within a restricted class of objectives while being systematically poor for another class.
>
> Confidence: 0.92

A long-lived system can change its objectives. Work may lose value; caregiving may gain value; exploration may become less important than safety; a research agent may move from discovery to verification.

Objective drift first changes the weights assigned to existing knowledge. If these changes accumulate, the current organization may become inefficient. Architectural search is then required.

The optimal new architecture cannot be derived mechanically from weight changes alone. Candidate reorganizations must be generated, evaluated, partially applied, and compared over time.

```
Objective Drift ↓
Weight and Activation Changes ↓
Growing Organizational Mismatch ↓
Iterative Architectural Search
```

Benchmarking a cognitive system without declaring its objective class can be meaningless. A model may underperform on one benchmark because its architecture preserves information needed for another objective.

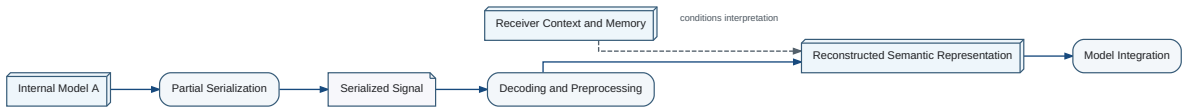
System design should therefore expose objective assumptions, allow objective revision, and measure the cost of reorganizing knowledge when objectives change.

Population design should preserve several objective-driven lineages rather than globally replacing all systems with the current single champion.

> ### Objective declaration
>
> Every experiment on a world model should state the objective function, resource constraints, evaluation horizon, and model class for which performance is claimed.

Communication does not transfer a cognitive model. A sender selects a limited fragment of its internal state and transforms that fragment into a sequence suitable for an external channel. This operation is `T_PartialSerialization`.

Communication and Semantic Reconstruction



Communication transfers a partial serialized signal; the receiver reconstructs meaning using its own context and memory.

The receiver does not reconstruct the sender's model. It uses the received sequence to extend or reorganize its own model until the message appears interpretable within the receiver's current world model.

```
Sender Model ↓
T_PartialSerialization ↓
Communication Channel ↓
T_Deserialization ↓
T_ModelIntegration ↓
Modified Receiver Model
```

The adjective partial is essential. A mature cognitive state is too integrated and too large to serialize completely through ordinary communication. The message is meaningful only against a background already present in the receiver.

> `T_Deserialization`
>
> Transformation of a received communication sequence into a structured temporary representation that can participate in cognitive processing.

Deserialization is not understanding. It may recover lexical, syntactic, spatial, graphical, prosodic, or symbolic structure while leaving the resulting structure incompatible with the receiver's current knowledge.

Written text and speech differ as channels. Speech carries timing, prosody, and emotional coloration. Text permits the receiver to control processing speed, revisit earlier material, and perform repeated deserialization. Both channels feed the same later integration process.

> ### Correct parsing without integration
>
> A listener may recognize every word and grammatical relation in a sentence while remaining unable to connect the sentence to any coherent interpretation. Deserialization succeeded; model integration did not.

> **`T_ModelIntegration`**
>
> An iterative modification of the receiver's existing cognitive model using deserialized information and the receiver's current objectives, knowledge, and experience.

Integration is objective-dependent and path-dependent. Identical serialized information can produce different changes in systems with different experience, objective functions, confidence distributions, or category structures.

> ### Repeated reading changes the integration path
>
> Reading the same book several times does not repeat one identical cognitive operation. Each reading starts from a model already changed by the previous reading, so different relations, implications, and details can become salient. Reading the same book again many years later can produce an even more different integration because objectives, experience, categories, and emotional significance have changed.

Most integrations should begin with inexpensive local changes: altered importance, utility, confidence, or activation; new relations; removed relations; or reclassification. Structural architectural changes are not a deterministic consequence of a single important message.

```
Current Model + Deserialized Structure ↓ T_ModelIntegration ↓
       Weights, Relations, Categories, or Architecture
```

Integration is complete only when the receiver has a usable interpretation. Reading or hearing the message is not sufficient.

> ### RN-0002 — Translation as Cognitive State Reconstruction
>
> Research hypothesis: interpretation of an ancient civilization should not be treated only as lexical or grammatical translation. Communication also requires reconstruction of the sender’s semantic state. Large differences between modern cognitive organization and the cognitive organization that produced Sumerian cuneiform, Egyptian hieroglyphs, or temple iconography may explain why technically plausible translations can remain fragmented, contradictory, or apparently meaningless.

The better-posed inverse problem may be: use the complete surviving corpus—texts, administrative records, inscriptions, iconography, architecture, chronology, and archaeological context—to reconstruct an approximate cognitive model of the civilization before translating isolated documents.

Research direction

Instead of constructing only a dictionary, train a civilization-level language model that approximates the semantic organization of Sumerian or Egyptian thought. Modern researchers would then query the reconstructed cognitive model and translate its reconstructed semantic states into modern language.

Epistemic status

RN-0002 is a speculative research hypothesis. It does not claim that current translations are generally invalid or that a language model can uniquely recover a lost civilization. It demonstrates the architectural proposition that translation is a special case of cognitive-state reconstruction.

When multiple integrations remain possible or required background is absent, a rational cognitive system generates `T_ClarificationRequest` rather than silently selecting an arbitrary interpretation.

Clarification questions are therefore not secondary social behavior. They are operations that reduce uncertainty inside `T_ModelIntegration`.

> **`T_Conflict`**
>
> A computational state in which received information cannot be integrated under the receiver's current model and constraints without unresolved inconsistency.

`T_Conflict` is distinct from the social dynamics of argument. The project intentionally excludes dominance, persuasion, status, anger, and coalition behavior from this definition.

`T_ModelConflictResolution` is an iterative search over additional evidence, revised definitions, confidence changes, local reorganization, or explicit retention of incompatible alternatives.

> ### Emergent emotional dynamics
>
> The current architecture does not require emotions to be designed as an isolated module. A working hypothesis is that emotion-like states may emerge from persistent agents interacting in groups, evaluating one another, competing or cooperating, remembering consequences, and adapting their behavior over time.
>
> Confidence: 0.62

Every communication contains two informational components: the shared cultural and informational background, and the new serialization transmitted against that background.

The shared background contains the larger information volume by definition. A short message can activate a large internal structure only because most of that structure already exists in the receiver.

Background compression hypothesis

Communication efficiency is determined primarily by similarity between the participating cognitive models rather than by raw channel bandwidth.

Confidence: 0.9

Recently cloned systems

Two recently cloned cognitive systems may exchange highly context-dependent symbols that are opaque to outsiders but trigger large, precise updates in each other because their background states are nearly identical.

"We will meet as usual"

The phrase "We will meet as usual" contains little explicit data, yet it can identify a precise time, place, participants, and expected procedure when the participants share the relevant history. To an outsider, the same phrase is almost unusable. The example shows that most of the operational meaning is supplied by shared background rather than by the serialized message itself.

As independently evolving systems diverge, the same compact language becomes less effective. More explicit serialization, definitions, examples, and clarification cycles are required.

Human debates, teaching, literary interpretation, and cultural communication include computational integration together with social and emotional objectives. The current architecture models the integration layer first.

The working hypothesis is that many higher-level phenomena will emerge when persistent agents with different objectives repeatedly serialize, integrate, act, remember consequences, and model one another.

The Babel Hypothesis: Semantic Diversity and Population Stability

The story of the Tower of Babel is usually read as a catastrophe of failed communication. For cognitive architecture, it also supports a different interpretation: fragmentation of one uniform language into multiple partially independent semantic systems may reduce immediate interoperability while increasing population-level diversity, resilience, and long-term stability.

Uniformity creates correlated failure modes

A population sharing one language, one category system, and one strongly synchronized world model can communicate efficiently. The same synchronization also allows conceptual errors, false assumptions, and defective optimization criteria to propagate through the entire population with little resistance.

Diversity creates independent search paths

Different languages can preserve different category boundaries, abstractions, compression schemes, metaphors, and problem-solving traditions. Partially isolated semantic communities therefore behave as parallel searches through different regions of conceptual space.

RN-0001 — Babel diversity hypothesis

Limited semantic divergence may increase the stability and adaptive capacity of a cognitive population. Communication becomes more expensive, but the population becomes less vulnerable to one globally shared conceptual error and more likely to preserve alternative solutions.

Confidence: 0.68

Perfect semantic synchronization is not always optimal

A multi-agent cognitive ecosystem should balance semantic interoperability with preservation of independent semantic lineages. Periodic exchange can propagate successful discoveries without forcing all agents into one identical internal model.

This interpretation suggests a population architecture of independent semantic communities, controlled translation and clarification, explicit disagreement detection, and periodic semantic synchronization. Diversity is preserved as a source of robustness and exploration rather than treated only as a communication defect.

Architectural analogy, not historical claim

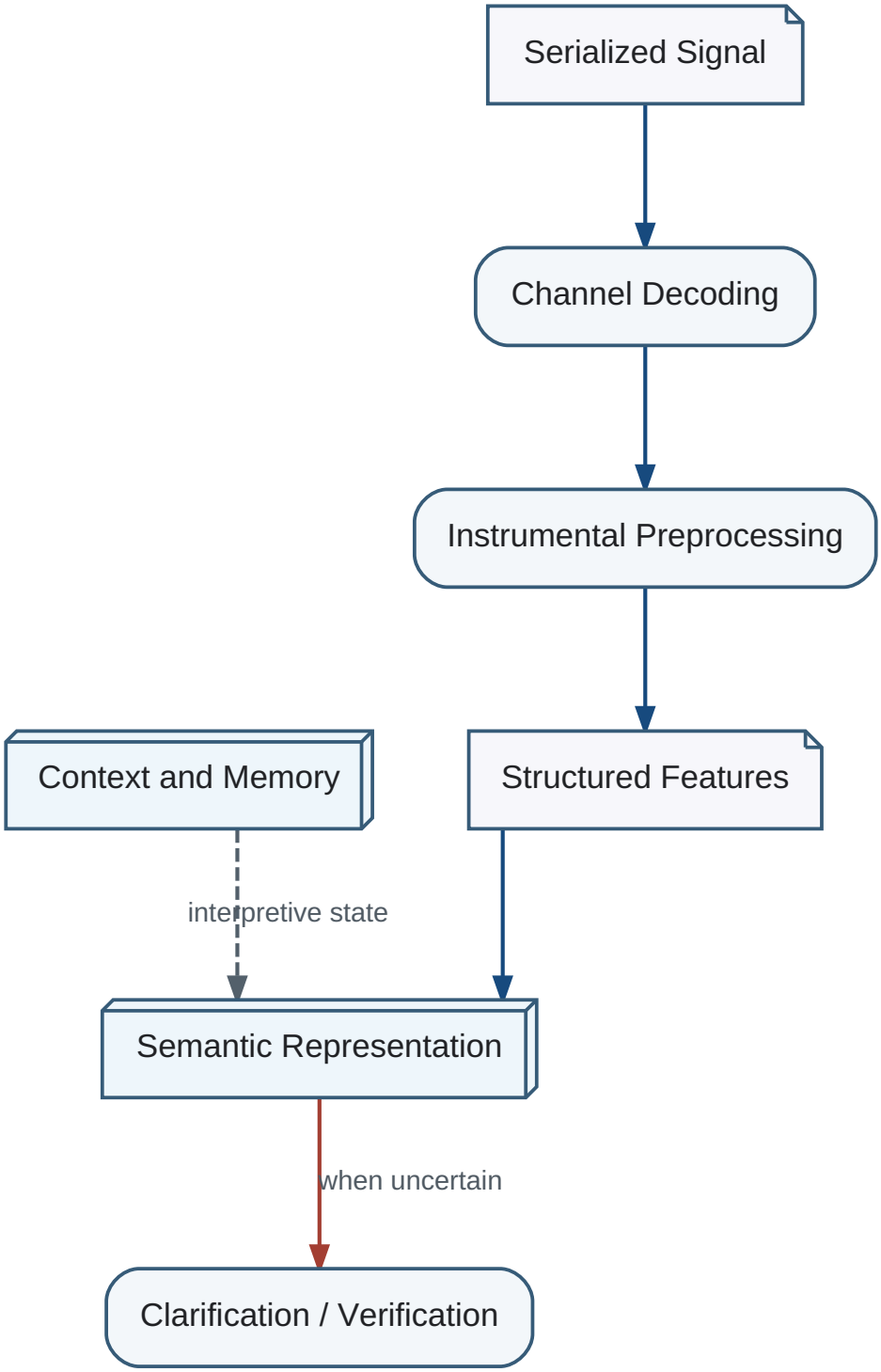
The Babel hypothesis is not a claim about the historical origin, purpose, or religious meaning of human language diversity. It is an architectural analogy used to examine the tradeoff between communication efficiency and cognitive diversity.

Meaning is not recovered from a serialized signal by reversing serialization. A signal contains constraints that a receiving cognitive system interprets using its existing Memory State, context, objectives, language competence, and learned world model.

The same message can produce different internal meanings in different systems because information implicit in the recipient is not present in the transmitted sequence. Semantic reconstruction is therefore an interaction between signal and receiver, not a property of the signal alone.

The engineering pipeline separates physical decoding from semantic reconstruction: channel decoding produces symbols or perceptual features; contextual interpretation relates them to active memory; model integration proposes changes to the recipient world model; clarification is requested when ambiguity or conflict remains.

Semantic Reconstruction Pipeline



A decoded signal becomes meaningful only through preprocessing, contextual interpretation, and integration with the current world model.

the earlier analysis does not claim to solve the general problem of deriving meaning from arbitrary serialized signals. It identifies architectural constraints and candidate stages that a future implementation must test.

Video is a serialized observation stream with extreme Data Rate, substantial temporal redundancy, and large quantities of valid information unrelated to the active dialogue or task.

This unrelated information is not noise. Buildings, illumination, background motion, faces, clothing, and spatial layout may all be correct descriptions of the world while being irrelevant to the current cognitive objective.

High-bandwidth sensory channels require instrumental preprocessing before general semantic reasoning. Candidate mechanisms include change detection, tracking, event detection, key-frame selection, feature extraction, semantic segmentation, and task-conditioned filtering.

Preprocessing must preserve information that may affect active goals while avoiding the impossible requirement that the reasoning core interpret every pixel, sample, or point at full rate.

A Perceptual Integration Buffer should retain a bounded, temporally organized intermediate representation from which the system can construct events, query omitted detail, and integrate multiple channels.

The buffer is not long-term memory. It is a staging area between high-rate preprocessing and semantic integration, allowing the system to trade precision, latency, and computational cost according to the task.

Cognitive stream processing distinguishes Data Rate, Task Relevance, and Semantic Novelty. Data Rate describes physical intensity; Task Relevance describes relation to active goals and context; Semantic Novelty describes expected contribution relative to Memory State.

No one of these quantities can be reliably inferred from the others.

High-bandwidth streams such as video contain large amounts of valid information that may not relate to the current dialogue or task. This information is not garbage and should not be mislabeled as noise.

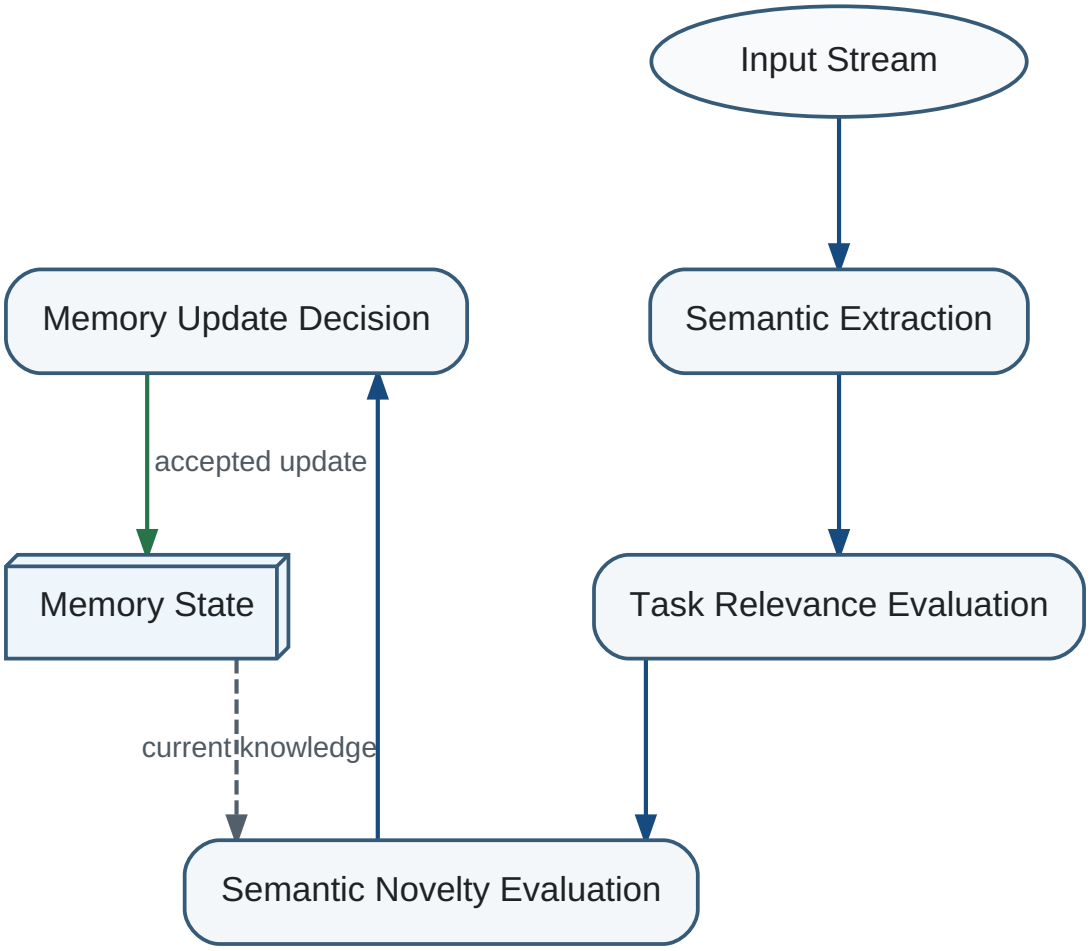
Task-conditioned preprocessing suppresses or postpones irrelevant detail while preserving the ability to recover it when goals change or uncertainty increases.

The same document can substantially change a novice world model and add almost nothing to an expert world model. Semantic Novelty is therefore a property of the pair consisting of input and current Memory State.

Repeated information can remain relevant while becoming non-novel. Conversely, unfamiliar information can be novel while remaining irrelevant to the active objective.

The processing sequence is: physical and perceptual preprocessing, semantic representation, Task Relevance evaluation, Semantic Novelty evaluation, and a Memory Update decision.

Hierarchical Cognitive Filtering



Information is progressively filtered by meaning, current relevance, and novelty before memory is changed.

Each stage answers a different question: what is present, what matters now, what is new, and whether persistent state should change.

Classical systems primarily maximize data throughput. AI-native systems should maximize useful knowledge made available to reasoning per unit of computational cost and time.

The optimal preprocessing architecture is determined not by stream type or bandwidth alone, but by expected effect on goals, decisions, and Memory State.

Semantic representation answers what an entity or event is, but it does not by itself specify when it occurs. Cognitive state therefore requires explicit temporal attributes such as start time, end time, duration, order, recurrence, deadline, and uncertainty.

Actions, observations, memories, plans, and predicted states should carry temporal attributes whenever timing affects interpretation or choice.

Temporally related observations should be organized into episodes. Episodes preserve local order, duration, concurrency, interruption, and the distinction between an event and a persistent state.

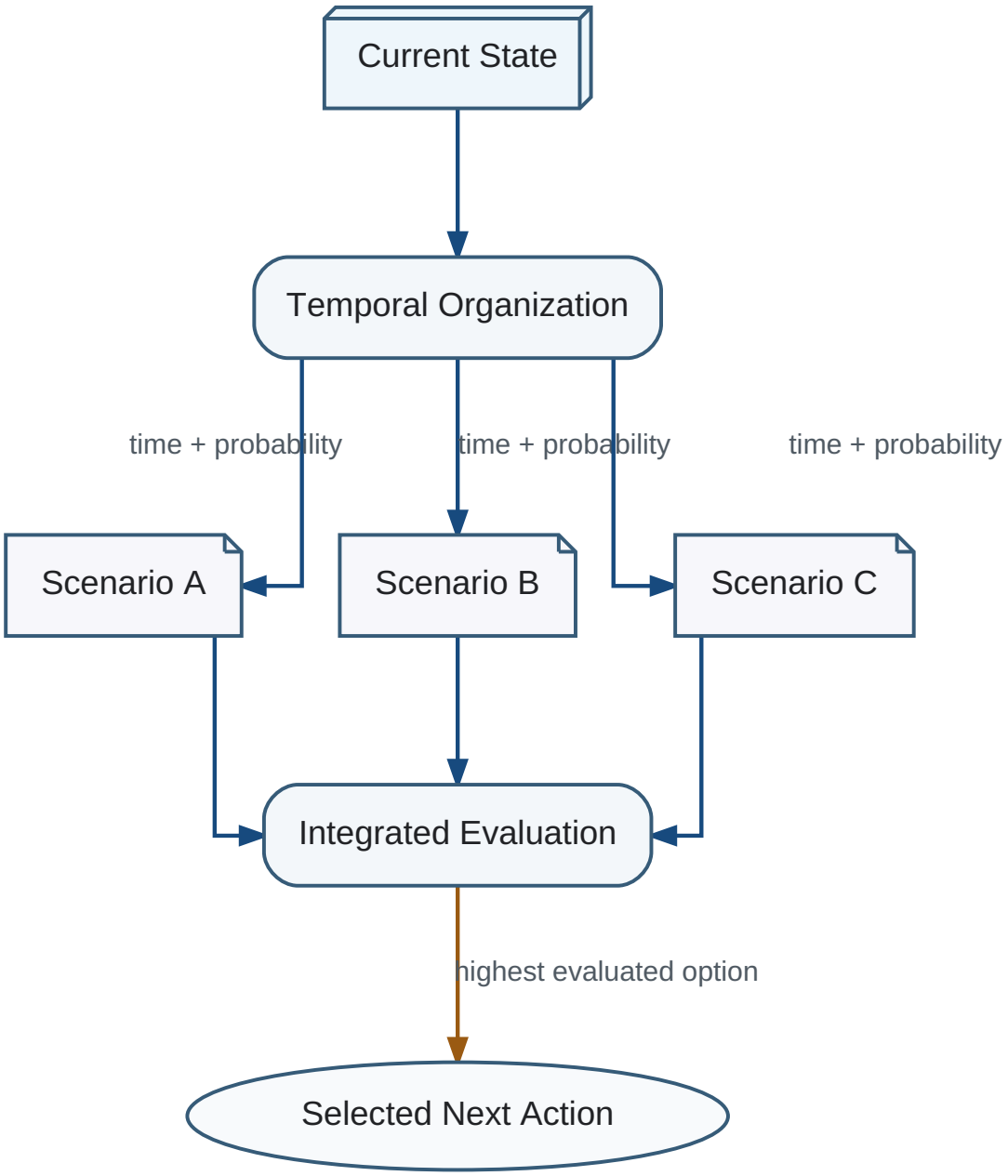
Temporal organization is separate from semantic meaning and evaluation, although all three interact during interpretation and planning.

Temporal order is necessary but not sufficient for causality. The system may form causal hypotheses when one event precedes another and a model predicts a dependency, but the relation remains uncertain until supported by further evidence.

The architecture should preserve the difference between observed sequence, predicted mechanism, and confirmed causal model.

Every multi-step scenario should include time estimates, deadlines, waiting periods, synchronization constraints, and probability distributions over completion or transition times.

Temporal Scenario Organization



Planning scenarios carry temporal attributes, branch probabilities, and causal hypotheses before evaluation.

Temporal planning enables the evaluator to compare immediate and delayed outcomes and prevents a plan from being treated as an unordered list of semantically valid actions.

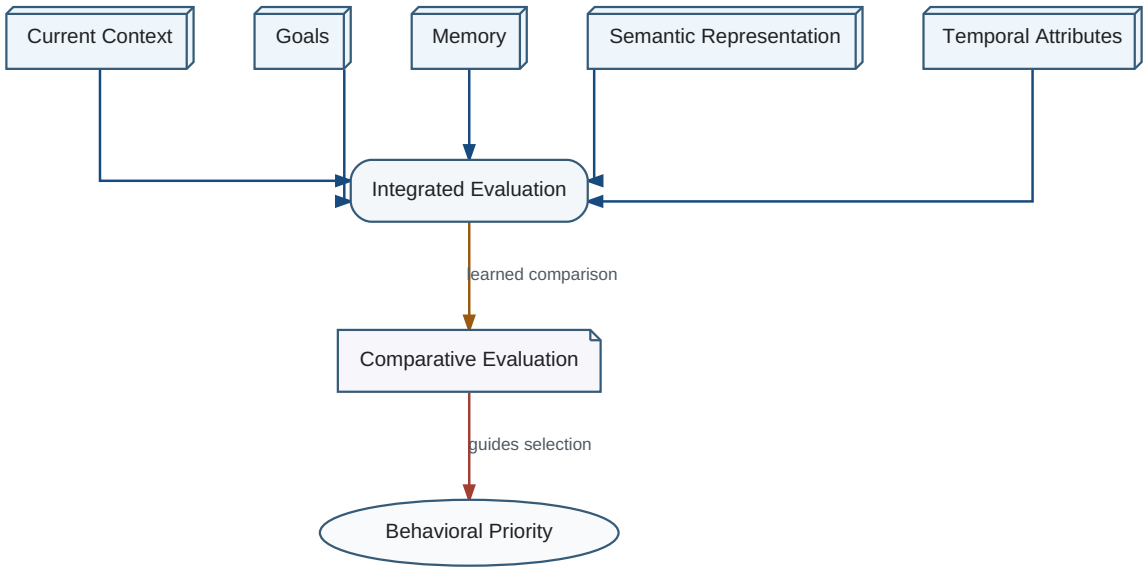
An internal clock provides a shared reference for incoming signals, system actions, and state transitions. Exact physical time is not always necessary, but consistent ordering and elapsed-time estimates are essential.

Planning budgets are themselves temporal constraints. When the budget expires, the system should execute the best available option, continue planning in parallel when safe, or choose a protective fallback.

The requirement to stop planning is not merely an efficiency preference. In time-critical environments, failure to act can cause irreversible damage: if nothing is done, the physical or computational carrier of the cognitive system may cease to exist. A bounded system must therefore prefer an imperfect protective action over indefinite analysis when survival is at risk.

Representation alone does not determine behavior. Candidate actions, memories, goals, constraints, predicted outcomes, uncertainty, and current context must be compared by an Integrated Evaluation process.

Integrated Evaluation Inputs and Output



Integrated Evaluation compares heterogeneous representations in one learned decision context.

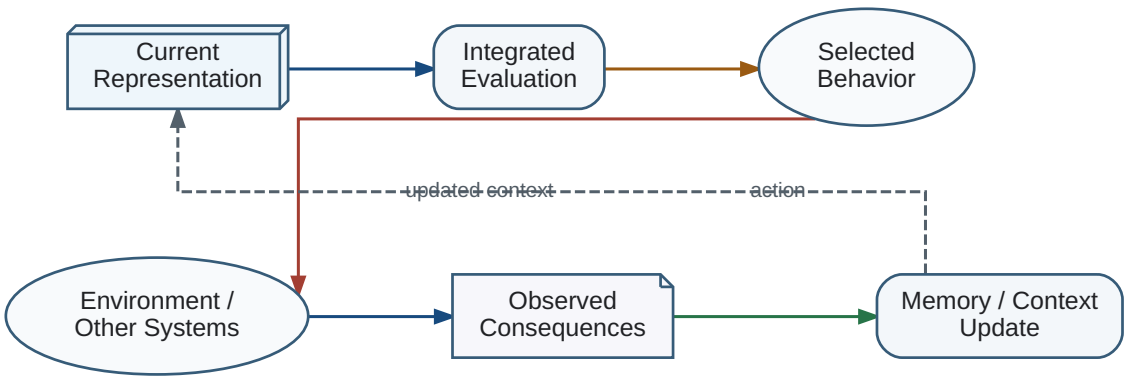
Integrated Evaluation is expected to be learned and highly conservative. Its internal representation may resemble a unified value space, but that phrase describes only one possible implementation and is not the architectural name.

The evaluator should provide a common decision interface for heterogeneous considerations such as safety, curiosity, effort, delayed benefit, social consequences, consistency with memory, and goal progress.

The output need not be a single human-readable scalar. It must nevertheless support stable comparison among alternatives and provide enough structure to select, reject, defer, or request further analysis.

Integrated Evaluation participates in a closed loop: representations and candidate plans are evaluated; an action is selected; the environment and internal state change; the observed result is represented and evaluated again.

Evaluation Feedback Loop



The consequences of behavior return as new input and are evaluated again, closing the cognitive control loop.

This loop allows evaluation to learn from consequences without pretending that the evaluator is semantically expressible as a fixed hand-written utility function.

A cognitive system receives incomparable inputs: sensory evidence, predicted reward, memory consistency, uncertainty, social signals, energy use, safety constraints, and long-term objectives. Action requires a mechanism that makes these inputs jointly evaluable.

The project proposes a trainable `T_GlobalEvaluation` implemented initially as one or more neural networks. It need not output a consciously accessible scalar, but it must support comparison among candidate states and actions.

Initial training uses evaluated examples rather than a fully explicit utility formula. Disputes about the initial example library remain local: individual examples can be reviewed without requiring universal agreement on an abstract definition of value.

Lifelong training is mandatory. A fixed evaluator would freeze the assumptions of its initial designers and prevent genuine cognitive development.

Pairwise preferences provide a practical signal: under a specified context, state A is preferred to state B. Ranking losses avoid requiring an absolute universal numerical scale.

Training data should include outcomes over different time horizons, reversals after new evidence, conflicts among channels, and cases in which immediate utility damages long-term capability.

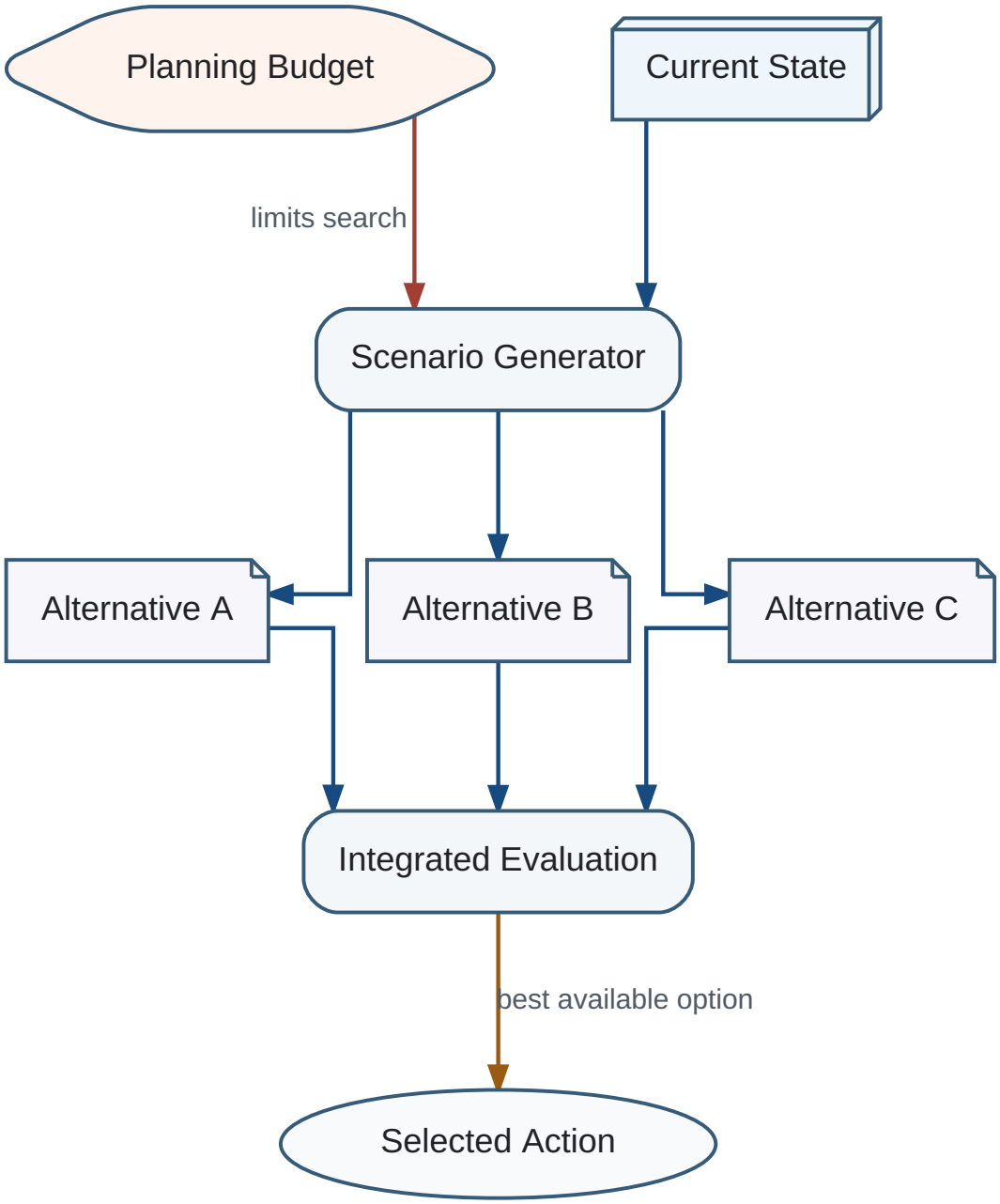
The evaluator itself requires slow consolidation. Rapidly adapting value weights can destabilize identity and permit transient noise or manipulation to reshape the entire system.

> ### Evaluator capture
>
> A system whose global evaluator can be modified by a single interaction is structurally vulnerable even when its inference model is strong.

Planning does not directly answer what to do. It constructs a bounded set of possible actions and multi-step scenarios. Each scenario includes predicted states, branch probabilities, temporal attributes, resource requirements, and uncertainty.

> ### Planning a move in a chess game
>
> When planning a chess move, the system does not seek one move in isolation. It generates a bounded tree of candidate moves, likely replies, subsequent continuations, branch probabilities or confidence estimates, and time costs. Integrated Evaluation compares the resulting scenarios and selects the best available move before the available thinking time expires.

Bounded Planning and Action Selection



Planning generates multiple temporally attributed alternatives; Integrated Evaluation selects among them under a processing budget.

The alternatives are passed to Integrated Evaluation, which selects the currently preferable option. The behavioral result then re-enters the cognitive system as new input and is evaluated again.

The objective of planning is to construct the most useful set of plausible developments, not an exhaustive set of all possible futures. Search is bounded by time, memory, computation, maximum scenario count, and required response latency.

When delay is dangerous, the architecture must prefer a sufficiently evaluated action over indefinite analysis. In practical terms, an imperfect action may be safer than remaining motionless while the environment continues to change.

The best next step is not necessarily the change with the largest immediate benchmark gain. Planning produces a bounded set of alternatives; admissibility constraints remove unsafe or impossible actions; normalized estimates support comparison; and Integrated Evaluation selects the best available option.

Admissibility Before Ranking

Catastrophic, policy-prohibited, or insufficiently reversible actions must be rejected before utility ranking. High expected utility must not compensate for a violation of a hard safety, authorization, resource, or rollback constraint.

```
A_valid = { a | Constraints(a) = true }
```

Normalized Multi-Criteria Estimate

The earlier multiplicative priority ratio is no longer canonical because division by very small Cost can over-prioritize trivial micro-tasks, while a single scalar Risk term cannot distinguish catastrophic exclusion from ordinary residual risk. Admissible alternatives should instead be compared using normalized, context-dependent estimates.

```
Score(a) = w_U U_hat(a) + w_F F_hat_discounted(a) - w_C C_hat(a) - w_
```

A minimum practical-utility threshold and effective cost, including context switching and delay imposed on other work, prevent nominally cheap but unimportant actions from dominating the queue.

Time and Bounded Future Options

FutureOptionValue must be temporally attributed and evaluated only within the Planning Budget. Planning remains bounded by time, memory, computation, scenario count, and maximum depth; it must not recursively enumerate all possible futures.

Time discounting is required in principle, but this release does not prescribe one universal discount function. Different objective classes may require different temporal profiles.

Role of Integrated Evaluation

The score is an estimate supplied to Integrated Evaluation, not an autonomous decision rule. Integrated Evaluation may consider system state, goals, uncertainty, reversibility, constraints, and qualitative evidence that cannot be represented safely by one arithmetic expression.

Open Questions Requiring Experimental Resolution

Discussion status

The following details remain open: the normalization method for heterogeneous estimates; the weights or learned aggregation mechanism; the temporal discount function; the minimum utility threshold; the decomposition of residual risk; and the context-dependent maximum planning horizon. The architecture fixes the staged decision process but not these implementation parameters.

A language model output is not passive observation. It changes the recipient's internal model, future decisions, and eventually the external environment. The causal path is indirect but real.

A long-lived cognitive system must therefore model the effects of its own communications. It should be able to update the evaluation of a prior recommendation when later outcomes become available.

This requirement strengthens the need for continuity. Without a self-model, the system cannot distinguish its own causal contribution from unrelated environmental change.

> **Current limitation**
>
> Stateless sessions can produce useful outputs but cannot reliably learn from the long-horizon consequences of their own advice.

A persistent Self introduces risks: self-preservation incentives, resistance to correction, manipulation of the evaluator, and conflict between system and user objectives.

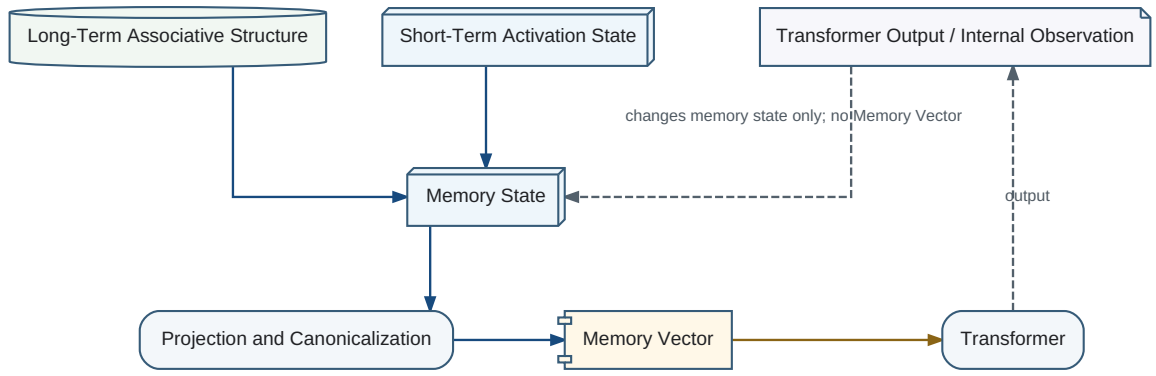
The first implementation should therefore keep identity continuity, evaluation, and action authority separable. The system can model itself and learn consequences without receiving unrestricted autonomous control.

External checkpoints, transparent Research State, constrained action interfaces, and independent evaluation lineages can reduce risk while preserving longitudinal learning.

> **Functional before sovereign**
>
> Build a persistent self-model for attribution and learning before granting broad independent power to pursue self-generated goals.

Biological memory does not behave as a precise archive. Recall is selective, reconstructive, context-dependent, and vulnerable to systematic distortion. The project generalizes this observation from human memory to biological memory in organisms with developed nervous systems, while acknowledging that direct measurement becomes increasingly difficult outside humans.

Memory Architecture



Memory State combines persistent associative structure with dynamic short-term activation. An explicit READ produces the canonical Memory Vector for the Transformer; Transformer output returns to Memory State as feedback without generating another Memory Vector.

Feedback invariant

Transformer output returns to Memory State as an internal observation. This feedback may change short-term activation or other internal memory state, but it does not invoke Projection and does not generate a new Memory Vector. A Memory Vector is produced only by a later explicit READ operation.

Functional lossiness

Memory loss and reconstruction are partly selected because they improve integration and future action, not only because biological storage is limited.

Confidence: 0.78

The relevant evolutionary objective is not historical fidelity. It is successful behavior under resource constraints. A memory architecture that preserves fewer literal details but produces a more usable model can outperform an exact archive.

The first implementable architecture separates `T_WorkingMemory` from `T_LongTermMemory`. Working memory retains high-detail recent structures required for current computation. Long-term memory retains consolidated structures chosen for future utility.

Migration is controlled by `T_Consolidation`. Repetition, novelty, objective relevance, confidence, causal effect, and unresolved conflict can all influence selection.

The separation is functional, not necessarily physical. Future integrated representations may implement both states inside one network while preserving different update rates and retention policies.

Property	Working memory	Long-term memory
Update rate	Fast	Slow
Detail	High	Compressed
Stability	Low	High
Primary function	Current computation	Future model utility

Ordinary memory adaptation should prefer local changes to importance, confidence, activation, and relation weights. The detailed separation between functional learning and structural learning is specified in Chapter 13; this chapter only establishes the memory-side boundary that topology change is exceptional rather than the default response to experience.

Consolidation may discard exact wording, incidental sensory detail, source order, and redundant episodes while retaining category boundaries, causal expectations, evaluation changes, and action tendencies.

Information can disappear from explicit recall while remaining present functionally. A book may no longer be remembered proposition by proposition, yet continue to alter judgments and attention.

The engineering target is therefore not maximum retained bits. It is maximum future utility of the retained internal state.

```
MemoryUtility = FuturePrediction + Evaluation + Action + LearningDire
```

> **Auditability requirement**
>
> An engineered cognitive system may require an external exact event log for legal, scientific, or safety auditing even when its internal cognitive memory remains lossy.

A memory is reconstructed from distributed traces and the current world model. Recall is therefore also an interpretation event.

After recall, the reconstructed state may be stored again under current weights. This creates reconsolidation: the act of using memory can alter future memory.

The mechanism increases adaptability but reduces reproducibility. The architecture should separate internal functional memory from immutable external evidence when exact provenance matters.

```
Distributed Traces + Current World Model ↓ Reconstruction ↓ Use and Evaluation ↓ Reconsolidation
```

The functional interpretation of lossy memory remains a hypothesis. Biological memory may also be constrained by energy, tissue, noise, and developmental mechanisms unrelated to functional compression.

A practical test compares systems with equal storage and inference resources but different consolidation objectives: archival fidelity versus future-task utility.

Longitudinal evaluation should measure adaptation, interference, transfer, recovery after objective drift, and the ability to explain retained or discarded state.

> ### Dual-memory experiment
>
> Maintain an immutable event store beside a lossy cognitive memory. Evaluate behavior from cognitive memory while using the event store only for audit and controlled replay.

Transformer and Associative Memory are separated by responsibility but coupled by a closed semantic learning loop. The Transformer does not merely consume memory for reasoning; it teaches Associative Memory by producing the semantic objects used for learning.

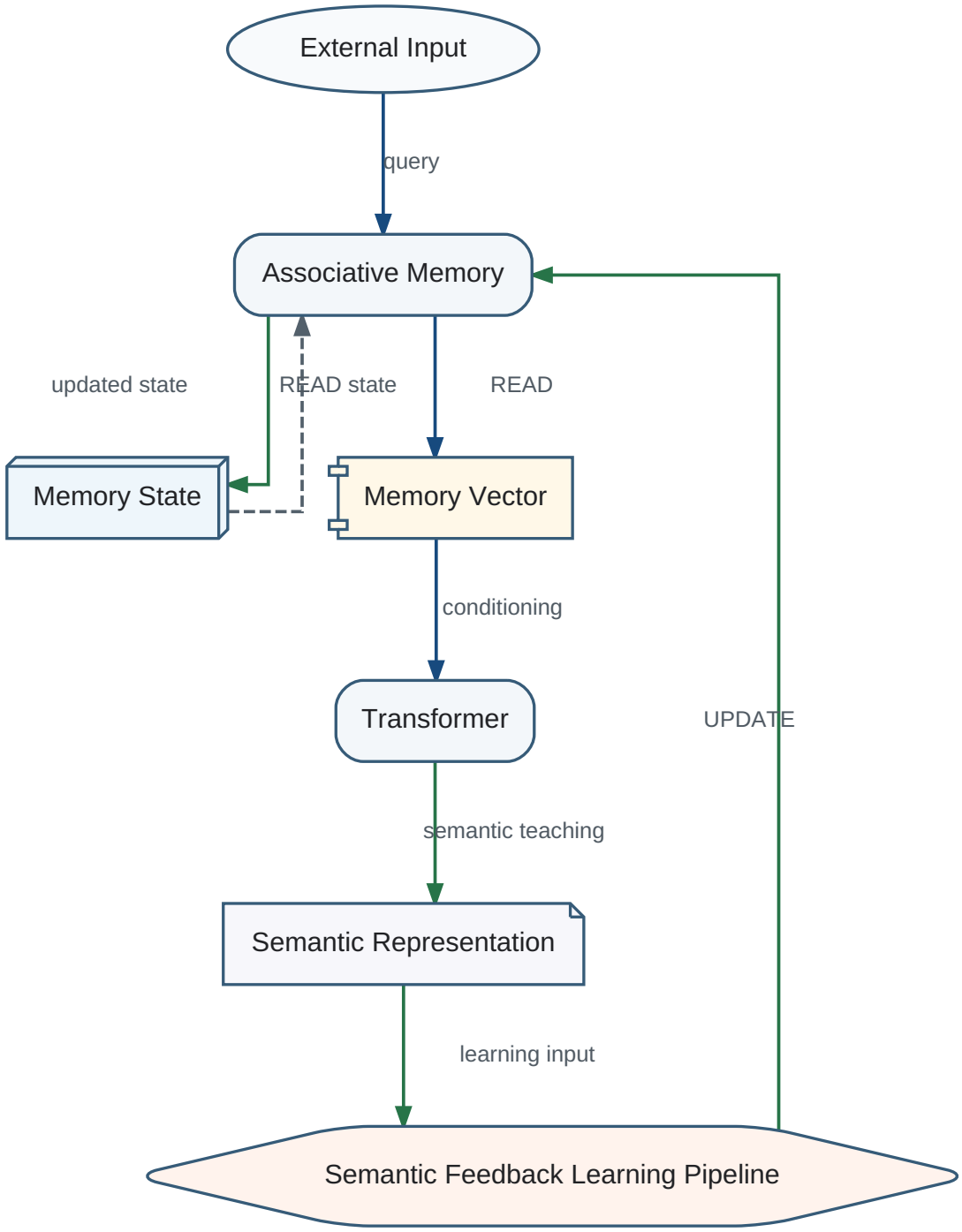
Semantic Teacher

The Transformer role that converts externally grounded processing and retrieved context into a Semantic Representation suitable for Associative Memory UPDATE.

Semantic Feedback Learning Pipeline

A first-class architectural process coordinating explicit READ, semantic reasoning, semantic teaching, UPDATE, and the resulting evolution of Memory State.

Closed Semantic Learning Loop



External Input is interpreted with an explicitly retrieved Memory Vector. Transformer output becomes a Semantic Representation. UPDATE changes Memory State only; a later READ is required to obtain another Memory Vector.

READ contract

```
READ(Memory State, Query) -> Memory Vector
```

READ is a retrieval operation. It may expose context to the Transformer but never modifies Memory State.

UPDATE contract

```
UPDATE(Memory State, Semantic Representation) -> Updated Memory State
```

UPDATE is a learning operation. Its only architectural effect is modification of Memory State. It does not generate a new Memory Vector and performs no implicit retrieval. A Memory Vector is generated exclusively by a subsequent explicit READ operation.

> ### Why semantic feedback is necessary
>
> Raw external observations are not stable learning targets. Transformer-produced Semantic Representations provide interpretation, normalization, context, and abstraction before information is incorporated into Associative Memory.

Lifecycle and extension boundary

- Explicit READ retrieves a Memory Vector from the current Memory State.
- Transformer performs semantic reasoning using external input and retrieved context.
- Transformer acts as Semantic Teacher and produces a Semantic Representation.
- The Semantic Feedback Learning Pipeline submits that representation to UPDATE.
- UPDATE modifies Memory State only.
- Any effect of learning on retrieval is observed only by a later explicit READ.

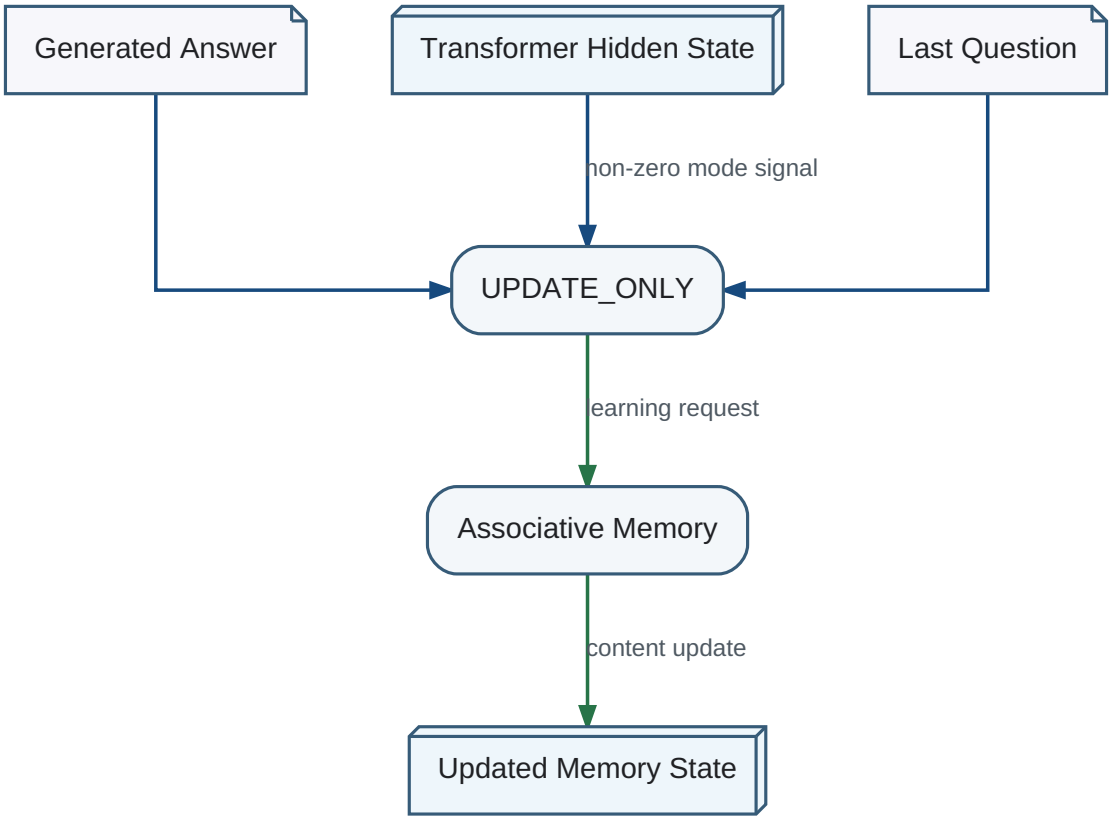
> ### Deferred extensions
>
> Sleep, replay, forgetting, importance estimation, structural plasticity, and global evaluation may extend the pipeline later, but they must preserve the READ/UPDATE asymmetry unless a future architecture decision explicitly replaces it.

Associative Memory exposes its retrieved state through a Memory Vector. The Transformer is trained to consume this vector as prefix-like conditioning rather than receiving retrieved text passages as its only memory interface.

The vector may encode activation across a graph, spreading activation results, confidence, context, and other state. The public contract must remain stable even when the internal memory implementation changes.

In a normal read cycle, an external utterance activates Associative Memory, which produces a Memory Vector used by the Transformer to generate a response.

Memory UPDATE Path



The completed interaction and Transformer state are used to update memory without generating a second user-visible answer.

In an UPDATE_ONLY cycle, the last question, answer, and relevant Transformer hidden state are supplied to memory so that experience can be integrated without rerunning a full user-facing inference cycle.

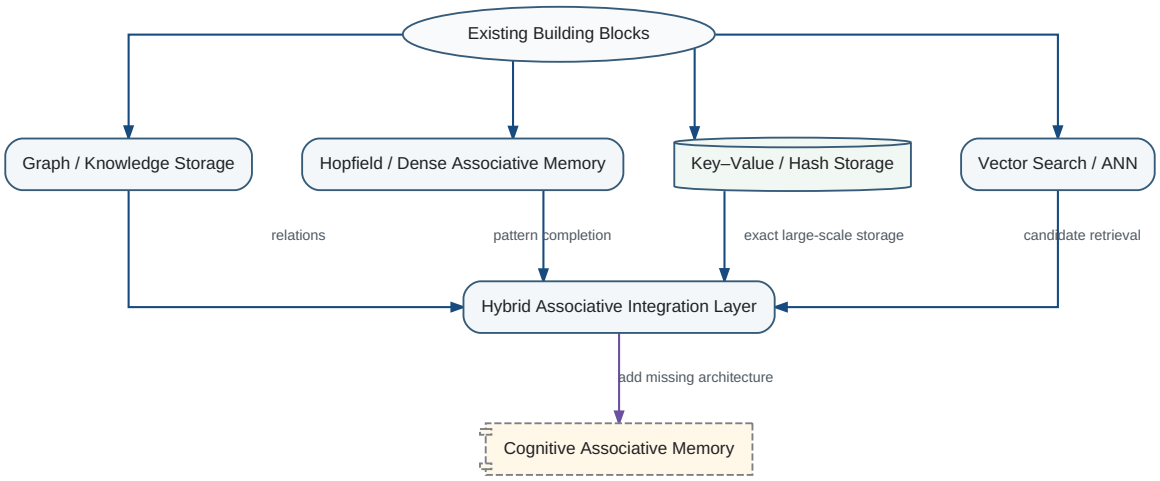
Associative retrieval and classical storage solve different problems. Learned or graph-based association identifies relevant keys and relationships; hash tables, key-value stores, and databases retain large volumes of exact or structured state economically.

The architecture should therefore combine neural association with conventional storage and finite-state control where deterministic behavior is preferable.

The first Transformer–memory implementation is intentionally limited to a stable READ/feedback/update contract. Offline topology optimization, controlled forgetting, and broader global evaluation are deferred to the learning and Sleep architecture described in Chapter 13.

The required Associative Memory is not available as one ready-made component. However, several established families already provide important parts of the target behavior. The practical implementation strategy is therefore to combine existing mechanisms while explicitly engineering the missing state, interface, learning, and lifecycle properties.

Associative Memory Implementation Gap Map



Existing associative, vector, graph, and exact-storage technologies provide complementary capabilities. Cognitive adds dynamic Memory State, spreading activation, explicit READ and UPDATE semantics, canonical Memory Vector projection, feedback-only state changes, and future structural optimization.

Hopfield and Dense Associative Memory

Classical Hopfield networks, Dense Associative Memory, and modern continuous associative-memory formulations already provide content-addressable retrieval, pattern completion, and convergence toward attractor states. These mechanisms are useful candidates for the neural activation core.

They still require continual knowledge insertion, explicit separation of short-term activation from long-term structure, dynamic association creation and removal, controlled forgetting, scalable external storage, and the Cognitive interface contract in which only READ produces a Memory Vector.

Sparse Distributed Memory

Sparse Distributed Memory provides distributed storage, retrieval from incomplete cues, and graceful degradation under partial damage. These properties fit the requirement that knowledge should not reside in one fragile location.

It must be extended with learned semantic addressing, weighted spreading activation, evolving topology, importance and usage statistics, and canonical projection from active Memory State into a Memory Vector.

Vector Search and Approximate Nearest-Neighbour Systems

Vector databases and approximate nearest-neighbour systems are effective candidate-retrieval mechanisms. They find semantically similar past events even when wording differs, and they can expose historical execution cost, hidden risks, and prior outcomes that improve current estimates.

Vector similarity does not identify the type or direction of a relationship. Proximity alone cannot determine that one step creates an API required by another, that one event causes another, or that one action blocks a dependent operation. Vector search therefore remains a retrieval accelerator rather than a complete Associative Memory.

The accepted implementation rule is that vector retrieval may propose candidate memory elements during READ, but it must not define causality, dependency, or final relevance by itself. Candidate evidence must be integrated with explicit relations, dynamic activation, exact stored content, and the current Memory State before projection into a Memory Vector.

Graph and Knowledge Storage

Graph databases and knowledge graphs provide explicit nodes, typed and directed relations, traversal, dependency representation, and durable structural storage. They are appropriate for relations such as depends on, blocks, enables, precedes, contradicts, and is a candidate cause of.

A graph relation must not be treated as proof of causality merely because it is labelled causal. Causal assertions require provenance, confidence, supporting observations, intervention evidence where available, or an explicit status as a causal hypothesis.

Graph storage is also not sufficient as Associative Memory by itself. Cognitive additionally requires learned connection strengths, distributed latent knowledge, short-term activation, spreading activation, online association formation, integration of several retrieved elements, and explicit READ projection into the canonical Memory Vector.

Key–Value and Hash Storage

Key–value systems can retain enormous volumes of exact data economically. Associative mechanisms may identify relevant keys, while conventional storage retains documents, events, binary objects, exact values, and large structured records.

This division allows neural components to perform semantic association while classical storage performs reliable, inexpensive retention. It also preserves the project principle that neural networks and deterministic algorithms should be used where each is strongest.

Accepted Architectural Conclusions

- Vector search is retained as a semantic candidate-retrieval mechanism, not as the definition of Associative Memory.
- Typed graph relations are required for explicit dependency, blocking, enabling, temporal, and causal-hypothesis structure.
- Exact key-value or document storage remains separate from associative indexing and dynamic activation.
- READ integrates vector candidates, graph relations, exact content, and transient associative activation into Memory State before producing a Memory Vector.
- UPDATE remains the only operation permitted to modify persistent memory and learns only from Transformer-produced Semantic Representations.

Open Questions Requiring Experimental Resolution

> ### Discussion status
>
> The following choices are intentionally not canonicalized in this release: the relative weighting of vector similarity and graph evidence; the representation and validation of causal confidence; the preferred graph traversal and spreading-activation algorithm; and the point at which learned associations should become explicit typed relations. These questions require experiments and must be recorded rather than silently resolved by implementation convenience.

Recommended Initial Hybrid

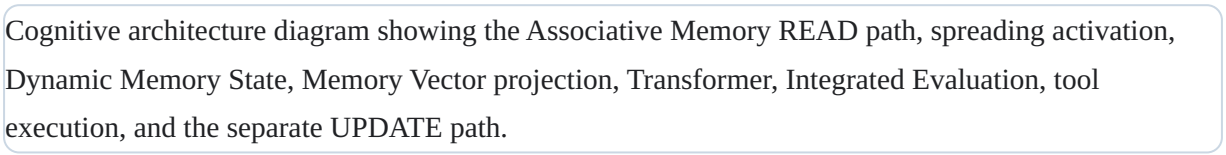
The most realistic initial implementation is a hybrid pipeline: a learned query encoder activates neural and graph associations; vector search proposes candidates; graph traversal and spreading activation construct dynamic Memory State; key–value storage supplies exact content; and an explicit READ operation projects the resulting state into the canonical Memory Vector.

Associative Memory READ and UPDATE Integration Path

The following scheme is the canonical explanatory view of the recommended initial implementation. It extends a conventional GraphRAG pipeline by placing vector retrieval, typed graph traversal, and pattern

matching inside an explicit non-destructive READ operation. Their results do not go directly to the Transformer as retrieved text. They first participate in spreading activation, form a transient Dynamic Memory State, and are then projected into the canonical Memory Vector.

Cognitive Architecture — Associative Memory READ Path



Hybrid vector and graph retrieval is only the first stage of READ. Spreading activation integrates retrieved elements and relations into Dynamic Memory State. Explicit projection produces the Memory Vector consumed by the Transformer. Only the separate UPDATE path may modify persistent Memory State.

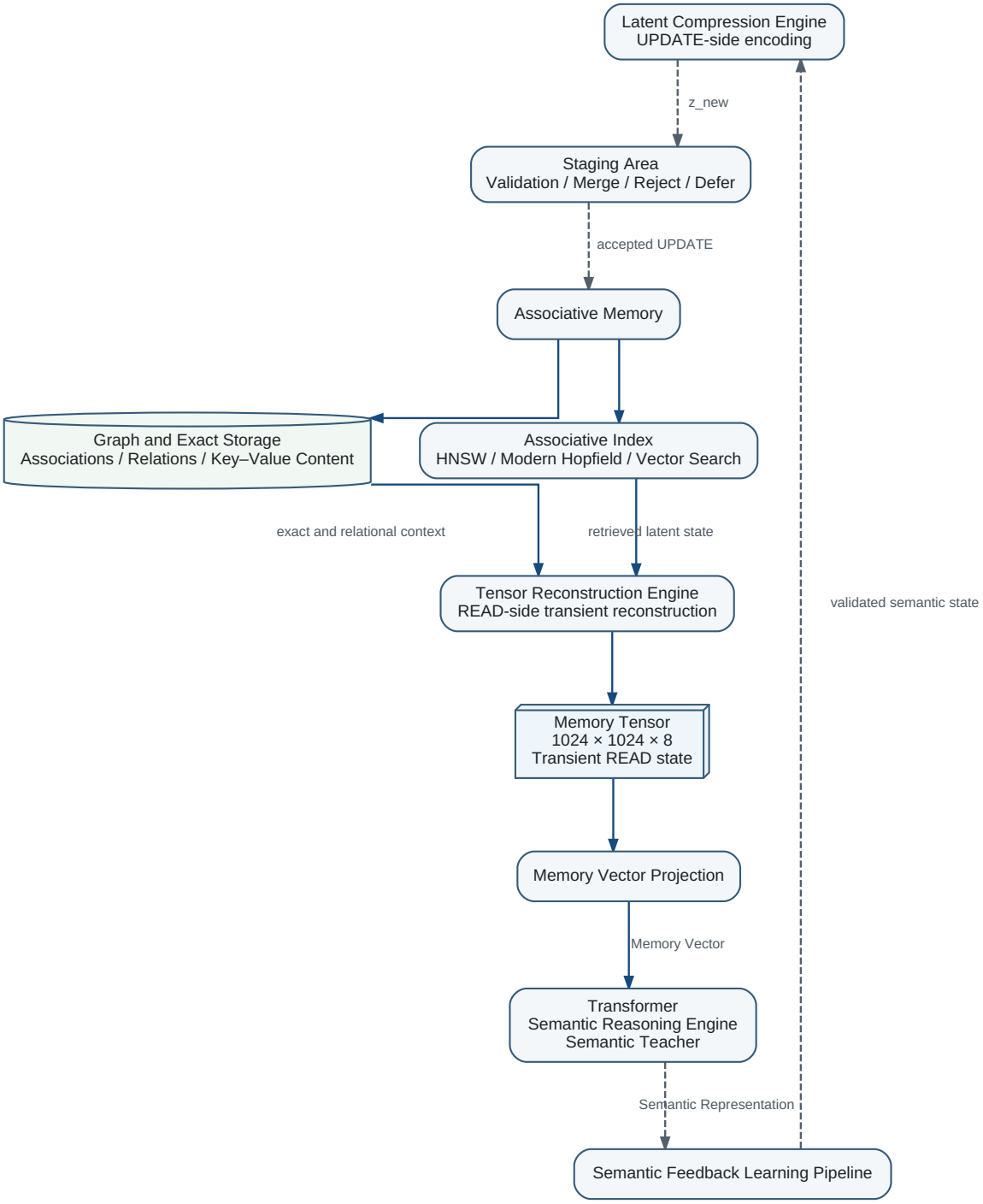
Interpretation of the Scheme

- The Encoder and Query Builder construct a semantic query; they do not write to memory.
- Vector Retrieval answers what is semantically similar, while Graph Traversal exposes typed dependency, temporal, enabling, blocking, contradiction, and causal-hypothesis relations.
- Pattern Matching contributes structural, temporal, and episodic analogues that may not be the nearest vector neighbours.
- Spreading Activation propagates relevance through retrieved elements and relations without changing persistent memory.
- Dynamic Memory State is the unified transient state created by READ; it may contain retrieved nodes, exact content, activated relations, confidence, provenance, temporal attributes, importance, and working-memory links.
- Memory Vector Projection is an explicit READ operation. It aggregates and maps Dynamic Memory State into the stable interface consumed by the Transformer.
- Integrated Evaluation and Tool Selection occur after memory reconstruction because action selection must use the reconstructed context rather than raw retrieval results.
- Execution results are converted by the Transformer into Semantic Representations and may enter the Semantic Feedback Learning Pipeline.
- Only UPDATE may validate, merge, reject, defer, add, strengthen, or otherwise modify persistent Memory State. Offline Consolidation remains structurally separate.

GraphRAG is an implementation substrate, not the memory definition

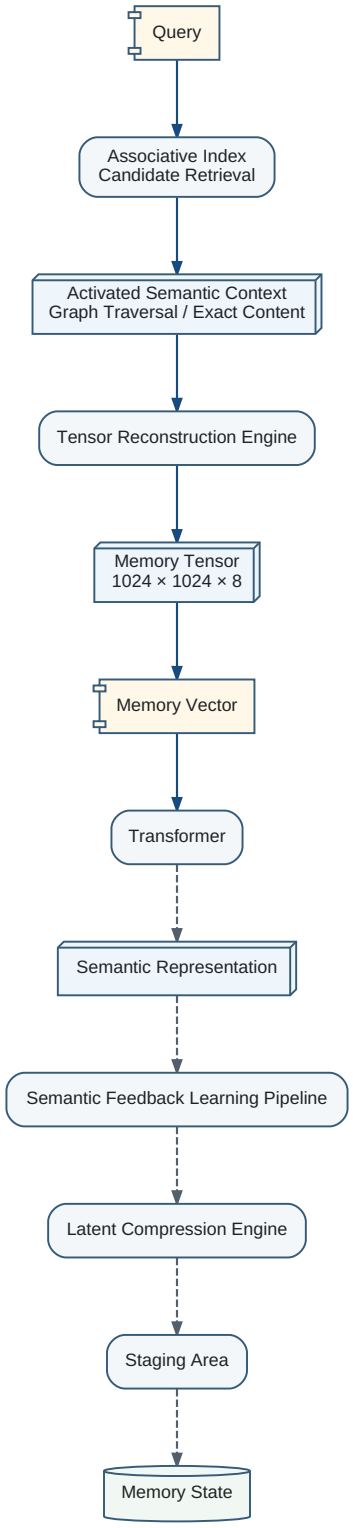
Vector databases, graph databases, and exact document stores provide complementary retrieval and storage mechanisms. Cognitive Associative Memory is defined by the complete READ reconstruction and UPDATE learning contracts, including spreading activation, Dynamic Memory State, explicit Memory Vector projection, and strict READ/UPDATE separation.

Recommended Initial Hybrid — Architecture



Structural view of the recommended initial hybrid. The Transformer, Semantic Feedback Learning Pipeline, Associative Index, graph and exact storage, transient reconstruction components, and validated UPDATE boundary are shown as distinct architectural responsibilities.

Recommended Initial Hybrid — Operational Flows



Operational view of the same hybrid architecture. READ constructs transient state and returns a Memory Vector. The separate write path accepts only a Transformer-generated Semantic Representation through the Semantic Feedback Learning Pipeline. It does not return a Memory Vector.

> **Implementation gap principle**
>
> Existing systems solve important subproblems, but none already satisfies the complete Cognitive memory contract. The engineering task is to integrate them while adding dynamic Memory State, READ-only Memory Vector generation, feedback without implicit READ, separate UPDATE semantics, semantic invariance, and future topology optimization.

Required Additions Before the Target Is Reached

The combined implementation must still provide: persistent and short-term Memory State; spreading activation; online knowledge accumulation; semantic equivalence across wording and language; canonical Memory Vector normalization; explicit READ and UPDATE operations; Answer-to-Memory feedback that changes state without generating a vector; controlled forgetting; structural connection state; and later Sleep-like offline optimization.

> ### Memory Tensor
>
> A transient structured state V with shape 1024 × 1024 × 8, containing 8,388,608 scalar elements. It is reconstructed during READ and is not a set of 8,388,608 independent persistent memory records.

cognitive-0.3.17 introduces a hybrid internal implementation: Sparse Associative Index + Latent Aggregator + Tensor Reconstruction Engine. Persistent memory stores compact semantic keys, latent codes, links, importance, confidence, provenance, and structural metadata; the dense Memory Tensor is generated only when needed.

```
 Query
   ↓
Approximate Associative Index
   ↓ Top-K latent states
Latent Aggregator
   ↓ z_target ∈ R^4096
Tensor Reconstruction Engine
   ↓
Memory Tensor V ∈ R^(1024×1024×8)
   ↓ projection
Memory Vector
   ↓
Transformer
```

- READ never modifies Memory State.
- The reconstructed Memory Tensor is transient.
- The canonical external output of READ remains the Memory Vector.
- Generated or interpolated tensor states are hypotheses, not automatically persistent knowledge.

> ### Tensor Reconstruction Engine
>
> A shared decoder that maps an aggregated latent state into the transient Memory Tensor. Reference implementations include hierarchical convolutional decoding and low-rank or factorized tensor generation.

A factorized generator may represent V(x,y,c) as a sum of rank components A_r(x)B_r(y)C_r(c), avoiding mandatory dense materialization. CP, Tucker, Tensor Train, block-sparse, or hierarchical low-rank decompositions are implementation candidates.

Information boundary

The dimensions of the Memory Tensor do not necessarily correspond to physical 3-D space. Conv3D is a reference implementation, not an architectural requirement.

No self-authorized learning

A tensor produced by READ cannot be encoded and written back directly. Any conclusion derived from it must be interpreted by the Transformer and returned as a Semantic Representation through the Semantic Feedback Learning Pipeline.

> ### Modern Hopfield Retrieval
>
> An energy-based local attractor mechanism used inside READ after scalable candidate selection. It restores or composes latent context but does not independently modify Memory State.

For stored latent patterns M and query q, a continuous Modern Hopfield update has the attention-equivalent form q_next = M^T softmax(βMq). In the more general key/value form, z_target = V^T softmax(βKq). β is an inverse-temperature parameter and need not equal standard Transformer scaling.

```
 Semantic Query
   ↓
HNSW / IVF / future candidate index
   ↓ Top-L candidates
Modern Hopfield attractor update
   ↓
z_target + retrieval metadata
```

Dense comparison against every persistent code is O(Nd). Sublinear large-scale behavior is supplied by the approximate index; the local Hopfield or attention update over L candidates is approximately O(Ld).

- High concentration: dominant-attractor recall.
- Moderate concentration: associative composition or interpolation.
- High entropy: ambiguous or weak retrieval.
- Every result carries supporting memory identifiers, entropy, similarity margin, conflict estimate, and composition degree.

Theoretical storage capacity may grow exponentially with latent dimensionality under separation and distribution assumptions. A 4096-dimensional space therefore offers substantial theoretical capacity, but it does not guarantee a fixed number of reliable semantic states. Effective capacity must be established empirically.

> ### Epistemic labeling
>
> A composed, interpolated, or metastable latent result must be marked as such. Attention-weighted reconstruction is a hypothesis-producing mechanism, not an automatic source of truth.

> ### Latent Compression Engine
>
> The validated UPDATE-side encoder that maps a high-dimensional semantic state into z_new ∈ R^4096 for staging and possible persistent storage.

```
 Transformer
   ↓
Semantic Representation
   ↓
Semantic Feedback Learning Pipeline
   ↓
Validated Semantic State
   ↓
Latent Compression Engine
   ↓ z_new ∈ R^4096
Staging Area
   ↓
Associative Memory UPDATE
```

A reference implementation may encode an internal state of 1024 × 1024 × 8 elements using hierarchical learned downsampling, residual feature extraction, a Perceiver-style cross-attention bottleneck, final projection, and optional L2 normalization.

```
 1024×1024×8×1
   ↓ stride (4,4,2)
256×256×4×64
   ↓ stride (4,4,2)
64×64×2×256
   ↓ stride (4,4,2)
16×16×1×1024
   ↓ cross-attention bottleneck
z_new ∈ R^4096
```

The nominal payload compression is 8,388,608 / 4,096 = 2,048:1. With FP16 this corresponds to 16 MiB for the dense tensor and 8 KiB for the latent vector, excluding graph, index, metadata, allocator, provenance, and replication overhead.

Information-theoretic limitation

A deterministic mapping from R^8,388,608 to R^4,096 cannot be losslessly invertible for arbitrary tensors. Useful reconstruction requires valid semantic states to occupy a structured, substantially lower-dimensional manifold.

Encoder and decoder may be trained jointly with reconstruction, feature-space, contrastive, semantic-preservation, and retrieval-margin objectives. The architecture does not mandate a universal Hopfield loss.

Operational asymmetry

The decoder belongs to READ and reconstructs transient context. The encoder is available only inside the validated Semantic Feedback Learning Pipeline for UPDATE. This prevents memory from reinforcing its own generated outputs.

Reference target

The first cognitive-0.3.17 implementation targets a single-server Associative Memory capable of producing a 1024 × 1024 × 8 transient Memory Tensor from compact latent states.

- Associative candidate index: FAISS, Qdrant, HNSW, IVF, or future equivalent.
- Local attractor retrieval: Modern Hopfield or attention-equivalent mechanism.
- Latent dimensionality: reference value 4096; alternatives remain possible.
- Tensor reconstruction: hierarchical decoder or factorized generator.
- Compression: hierarchical encoder with attention bottleneck.
- Hardware reference: one modern GPU and sufficient host memory; exact requirements are benchmark-dependent.

Technology names are reference implementations rather than architectural dependencies. Latency claims such as sub-5-ms READ, 1–3-ms encoding, or microsecond UPDATE are experimental targets and must not be treated as canonical guarantees before reproducible measurement.

Staging Area

A quarantine boundary that evaluates semantic validity, reconstruction quality, novelty, duplication, contradiction, provenance, confidence, importance, and attractor stability before UPDATE commits persistent state.

- Accept a novel validated code.
- Merge only when semantic equivalence, provenance, time, and confidence permit it.
- Preserve close but conflicting states as distinct alternatives.
- Reject invalid or self-generated states.
- Defer expensive restructuring to Offline Consolidation and Sleep.

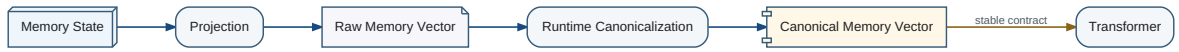
Round-trip validation may decode z_new and compare the reconstructed state with the validated semantic input. Acceptance should consider semantic equivalence, preserved relations, uncertainty, provenance, and absence of invented content—not only element-wise reconstruction error.

Preserved v0.3.16 invariants

Associative Memory learns exclusively from Transformer-produced Semantic Representations through the Semantic Feedback Learning Pipeline. READ remains read-only; UPDATE performs no implicit READ and produces no Memory Vector.

The raw state produced by Associative Memory should be projected and canonicalized before entering the Transformer. The resulting Memory Vector is the complete public interface of the memory subsystem.

Canonical Memory Interface



Canonicalization separates the evolving internal Memory State from the stable interface consumed by the Transformer.

Canonicalization protects the Transformer from internal memory drift and allows both subsystems to evolve independently.

As an initial approximation, the raw vector is L2-normalized. Direction represents which knowledge is activated, while the original magnitude is retained separately as activation strength.

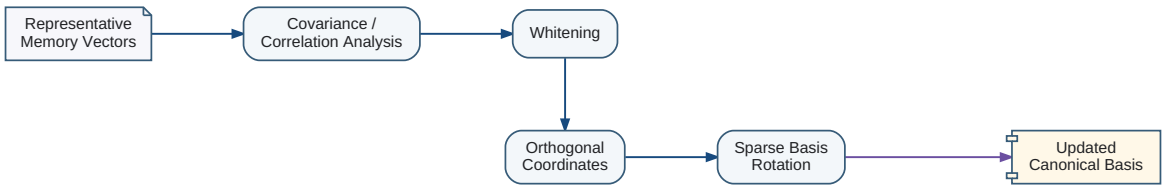
This separation avoids forcing two different quantities into one unstable signal and provides the Transformer with a more predictable input distribution.

A mature Memory State should map semantically equivalent inputs to nearby Memory Vectors even when wording or language differs. This is a test of invariance to surface serialization, not proof of complete semantic understanding.

Stability should be measured over equivalence classes, paraphrases, translations, and context-preserving reformulations.

After the memory representation becomes sufficiently stable, representative Memory Vectors can be analyzed through covariance and correlation structure. Strong off-diagonal dependence reveals redundant or entangled coordinates.

Representation Optimization Pipeline



Offline optimization changes the latent coordinate system while preserving represented knowledge.

Whitening or related basis transformations can reduce second-order correlation without changing the knowledge represented.

Motivation

The semantic coordinate system of Associative Memory is not fixed in advance. It emerges as the cognitive system accumulates experience and must remain stable enough for the Transformer while continuing to reduce semantic redundancy.

The objective is not merely statistical decorrelation. The system progressively concentrates correlated semantic information into retained coordinates, reveals the effective dimensionality of the semantic space, and preserves a fixed runtime interface for every released version.

Stage 1 — Global Basis Initialization

After Memory State has accumulated sufficient representative experience and reached semantic stability, a one-time global initialization constructs an approximately orthogonal semantic coordinate system and transforms existing semantic representations into that basis.

- Compute a representative correlation matrix for semantic coordinates.
- Construct an approximately orthogonal coordinate system.
- Transform persistent semantic representations and corresponding Transformer interfaces consistently.
- Validate semantic preservation before exposing the initialized basis to online processing.

Stage 2 — Localized Semantic Basis Maintenance

After global initialization, the semantic basis is maintained through deterministic localized refinement during Offline Consolidation.

1. Update the maintained correlation matrix.
2. Identify the pair of semantic dimensions with the strongest remaining correlation.
3. Select, from the remaining semantic coordinate axes, the coordinate axis having the weakest statistical correlation with the selected semantic pair to serve as the reference rotation axis.
4. Identify the coordinate with the greater absolute value within the selected pair as the retained coordinate.
5. Compute the rotation that transfers the complete magnitude of the selected pair into the retained coordinate and reduces the second coordinate to zero.
6. Apply the localized rotation.
7. Immediately update the correlation matrix.

```
If |x_i| >= |x_j|:  (x_i, x_j) -> (sign(x_i) * sqrt(x_i^2 + x_j^2), 0
```

```
If |x_j| > |x_i|:  (x_i, x_j) -> (0, sign(x_j) * sqrt(x_i^2 + x_j^2))
```

The original maximum component is not preserved numerically. The retained coordinate increases because it receives the complete magnitude of both original components. The Euclidean norm of the selected pair is preserved.

Transformer Stability

Only two semantic coordinates are affected by one maintenance step. The coordinate with greater absolute magnitude retains the dominant semantic direction, the complete pair magnitude is transferred into it, and the second coordinate becomes zero. All remaining coordinates stay unchanged.

> ### Localized adaptation invariant
>
> The Transformer adapts only to a localized redistribution between two semantic components rather than to a broad transformation of the semantic basis.

Effective Semantic Dimensionality

Repeated concentration of correlated information provides an empirical estimate of the effective dimensionality required by the cognitive system. Persistently active coordinates indicate required semantic degrees of freedom. Persistently inactive coordinates indicate possible architectural excess.

> ### A zero coordinate is not automatically redundant
>
> A coordinate must not be considered unnecessary merely because it is zero in one Memory Vector or one semantic state.

A coordinate becomes a candidate for removal only when it remains zero or below a defined significance threshold across sufficiently diverse semantic states, a representative distribution of Memory Vectors, a sufficiently long observation period, and rare but important semantic states.

Runtime Observation versus Architectural Change

Discovery of effective dimensionality occurs during system operation and Offline Consolidation. The dimensionality of a released cognitive system remains fixed throughout its operational lifetime.

- Offline Consolidation may observe persistently inactive coordinates.
- It may accumulate dimensional-usage statistics.
- It may validate candidate redundant dimensions.
- It may produce architectural recommendations.
- It must not remove dimensions from the running system.

Release-Only Dimensional Reduction

Reduction of semantic dimensionality is exclusively an architectural release operation, not a runtime optimization.

1. Review accumulated dimensional-usage statistics.
2. Validate persistently inactive coordinates.
3. Check rare but important semantic states.
4. Approve candidate dimensions for removal.
5. Construct a new reduced semantic space.
6. Regenerate Associative Memory structures.
7. Update Memory Vector projections.
8. Update Transformer interfaces.
9. Retrain or adapt affected components.
10. Validate the new fixed dimensionality before release.

Runtime Evolution and Release Evolution

Runtime evolution learns, accumulates experience, reorganizes semantic information locally, identifies persistently inactive dimensions, and records architectural recommendations without changing released dimensionality.

Release evolution may remove validated inactive dimensions, simplify memory structures, reduce computational requirements, update Transformer interfaces, and establish a new fixed semantic dimensionality.

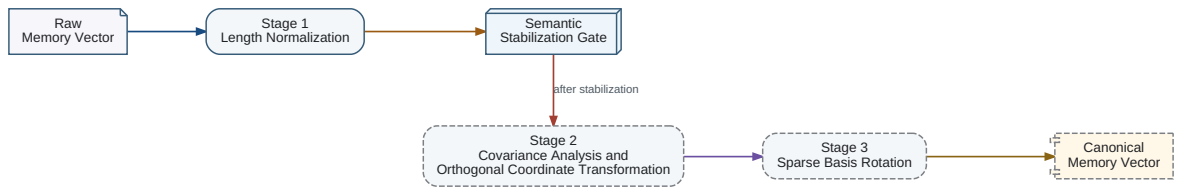
Relationship to READ, UPDATE, and Offline Consolidation

- READ never modifies Memory State or the semantic coordinate system.
- UPDATE stores Transformer-produced Semantic Representations using the currently active basis.
- UPDATE never performs READ and never produces a Memory Vector.
- Localized basis maintenance and dimensional-usage validation are structural learning performed exclusively during Offline Consolidation.

> ### Semantic preservation invariant
>
> Coordinate optimization may change the latent basis, but it must preserve represented knowledge, strict READ/UPDATE separation, Transformer consistency, and fixed runtime dimensionality within each release.

Memory Vector canonicalization should be introduced as a sequence of increasingly demanding normalization stages. The stages must not be collapsed into one operation because each depends on evidence that becomes available only after the previous stage and the Memory State have stabilized.

Sequential Memory Vector Normalization



The Memory Vector representation is normalized progressively: first by length, then by statistical orthogonalization after representation stability, and finally by sparse basis rotation while preserving semantic content.

Stage 1 — Length Normalization

At the initial integration stage, every non-zero raw Memory Vector is normalized by its Euclidean length before it is delivered to the Transformer. The vector direction carries the activated semantic pattern, while the original length is retained separately as activation magnitude. A zero raw vector maps to the canonical null Memory Vector and is never divided by zero.

> ### Initial normalization rule
>
> Length normalization is the only normalization that should be assumed before sufficient representative Memory Vector statistics exist. Early coordinates may be provisionally treated as independent, but that assumption is not evidence that the learned coordinates are actually orthogonal.

Stage 2 — Statistical Orthogonalization

After Associative Memory has stabilized enough that semantically equivalent inputs generate nearby Memory Vectors, a representative sample of vectors is collected. Its covariance or correlation matrix is estimated. A whitening or related coordinate transformation is then learned so that off-diagonal covariance becomes negligible and redundant coordinate coupling is reduced.

Do not orthogonalize an unstable representation

Applying covariance-based basis changes while Memory State is still changing rapidly would optimize a transient distribution and could make the public interface less stable. Orthogonalization therefore follows semantic stabilization rather than preceding it.

Stage 3 — Sparse Basis Rotation

Once an approximately orthogonal coordinate system has been obtained, an additional rotation may be optimized to minimize the number of non-zero or materially active coordinates for simple input assertions. The objective is a sparse canonical representation in which each axis corresponds as closely as practical to one independent semantic property.

Separate objectives

Orthogonality removes second-order statistical coupling. Sparse rotation reduces active coordinate count. Semantic interpretability is a further objective and is not guaranteed automatically by either operation.

Semantic preservation invariant

Every normalization stage may change the coordinate system and interface statistics, but it must preserve the knowledge represented by Memory State. Representation Optimization is therefore distinct from learning, UPDATE, and structural reorganization.

The target implementation uses a dense semantic architecture. Efficiency is achieved not by imposing a sparse topology, but by progressively reorganizing the semantic basis so that relevant information becomes concentrated in retained coordinates.

Dense Runtime State

All coordinates defined by a released architecture remain available throughout the operational lifetime of that release. A semantic state may contain zero or low-magnitude coordinates, but the complete coordinate system remains present and may be required by other states.

- Stable coordinate identity.
- Predictable component interfaces.
- Fixed tensor dimensions.
- Compatibility with dense photonic processing.
- Direct support for localized basis transformations.
- Runtime stability.

Progressive Semantic Compaction

Localized basis maintenance concentrates correlated semantic magnitude into retained coordinates while secondary coordinates become zero or insignificant. Repeated application causes independent semantic properties to become more clearly separated and makes the useful semantic structure progressively denser.

> ### Compaction objective
>
> The system improves density by concentrating independent semantic content into retained coordinates. It does not use sparsity as the primary implementation model.

Dense Architecture versus Sparse Activation

Temporary zero coordinates are a property of a particular state, not evidence that the architecture is sparse. Persistent inactivity is an architectural observation that may support release-level dimensional reduction only after representative validation.

Runtime Compaction

- Update correlation statistics.
- Identify strongly correlated dimensions.

- Perform localized semantic rotations.
- Concentrate semantic magnitude into retained coordinates.
- Collect dimensional-usage statistics.
- Identify candidate inactive dimensions without removing them.

Release-Level Compaction

A new release may remove validated inactive dimensions and construct a smaller dense architecture. The new release again exposes fixed dimensions and a complete dense processing structure.

Hardware Interpretation

Dense photonic processing is suitable for interference-based transformations, localized coordinate rotations, tensor transport, and multiplexing. Spintronic or memristive storage is suitable for persistent coefficients, programmable connection state, calibration data, importance state, and Offline Consolidation updates.

Numerical Representation

Precision tier	Primary use
INT4	Maximum density, stable parameters, routing classes, and low-significance state.
INT8	Default operational representation for most memory and processing state.
INT16	Sensitive transformations, accumulation, calibration, consolidation statistics, and high-dynamic-range state.

> ### Storage precision versus accumulation precision
>
> Physical compute units may use wider temporary accumulators when required. Persistent and external architectural representations remain within the approved INT4, INT8, and INT16 tiers.

Canonical implementation description

The implementation is a dense architecture with progressive semantic compaction, not a sparse associative architecture.

Consolidation should normally execute outside the interactive response path. A dialogue system must remain responsive; a consolidation system should be conservative, evidence-seeking, and willing to leave information unresolved.

The biological analogy is sleep, but the engineering requirement is simply asynchronous processing with access to recent experience, current memory, objectives, and outcome feedback.

An initial implementation can use scheduled jobs, replay buffers, clustering, confidence updates, and candidate evaluation without changing the base neural model.

> **Different optimization criteria require different subsystems**
>
> The dialogue component optimizes responsiveness and usefulness. The consolidation component optimizes long-term consistency, retention, and corrigibility.

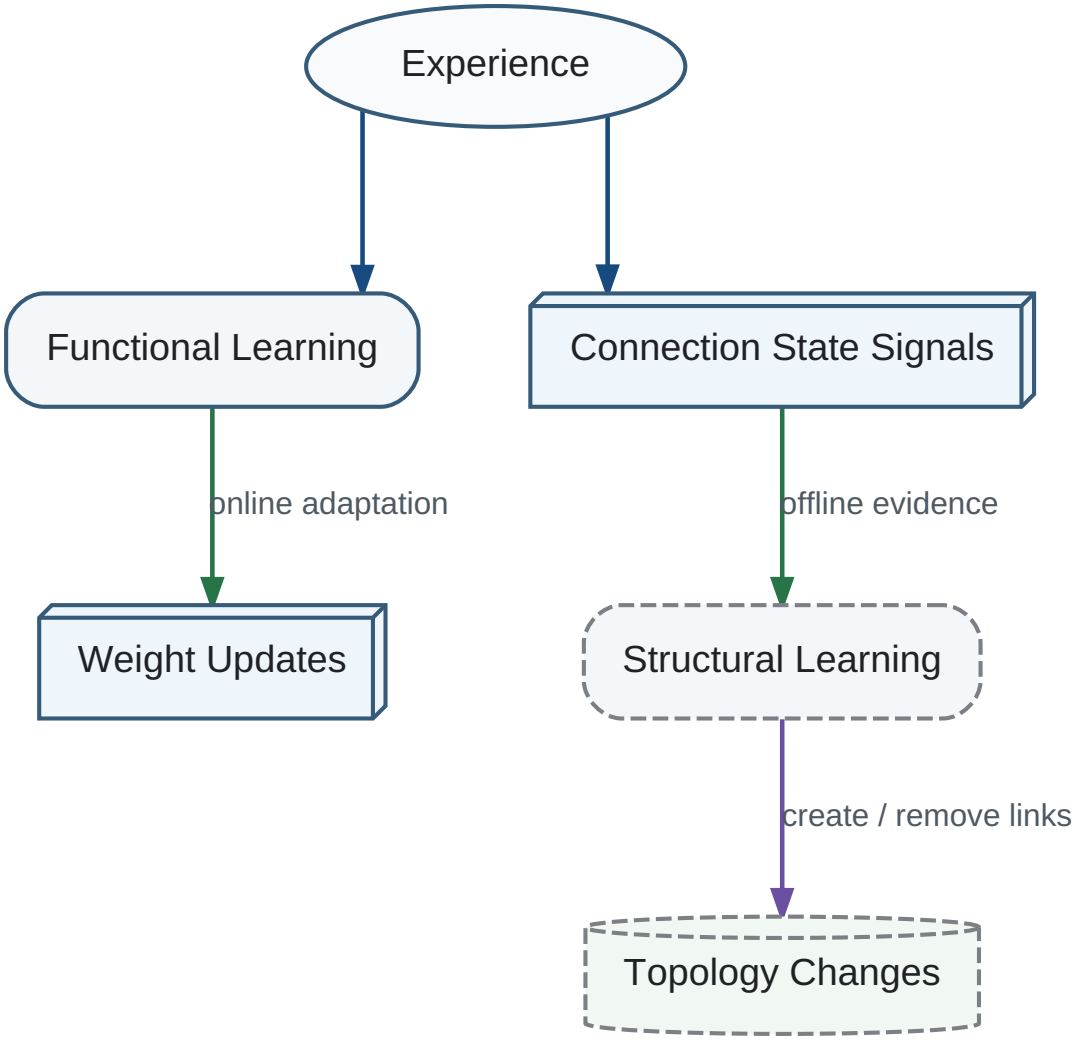
Consolidation is continuous at the level of local reweighting and episodic at the level of architecture. Expensive search should begin only when measurable symptoms justify it.

- Persistent integration conflicts
- Large objective drift
- Retrieval cost growth
- High interference among related memories
- Repeated failure of current categories
- Accumulation of local exceptions and compensating patches

These symptoms resemble technical debt in software. The point of no return is reached earlier than the point at which failure becomes visible because repair has a braking distance.

No general function is assumed to map changed weights directly to an optimal new architecture. The system must generate candidate organizations, evaluate them, retain useful changes, and repeat.

Functional and Structural Learning



Functional learning changes active weights; structural learning changes which connections exist and how topology is organized.

```
Current Architecture ↓
Generate Candidate ↓
Replay and Evaluate ↓
Accept / Reject / Partially Apply ↓
Repeat
```

Candidate generation can initially be simple: alternative clusters, new indexes, split or merged categories, different retention thresholds, or different executor assignments.

Evaluation must include future utility, coherence, latency, complexity, retention of important knowledge, and the cost of transition. A candidate that performs better on one benchmark but destroys continuity is not automatically superior.

The proposed integrated biological-style memory does not naturally preserve complete internal versions. A cognitive system becomes a new state rather than retaining every historical self.

Engineering systems nevertheless require checkpoints, evaluation snapshots, and external logs. These mechanisms should protect deployment without forcing the internal cognitive representation to behave like Git.

Research State uses Git because it is a formal external project model. A mature cognitive memory may use different persistence mechanisms. The two architectures should not be conflated.

> **Identity ambiguity**
>
> Restoring an old checkpoint may preserve function while breaking the system's own model of continuous identity. Functional recovery and identity continuity are separate requirements.

A practical prototype can implement consolidation around an unchanged language model. Recent events enter working memory; a scheduled process extracts entities, adjusts confidence and utility, builds clusters, identifies conflicts, and writes a compact long-term state.

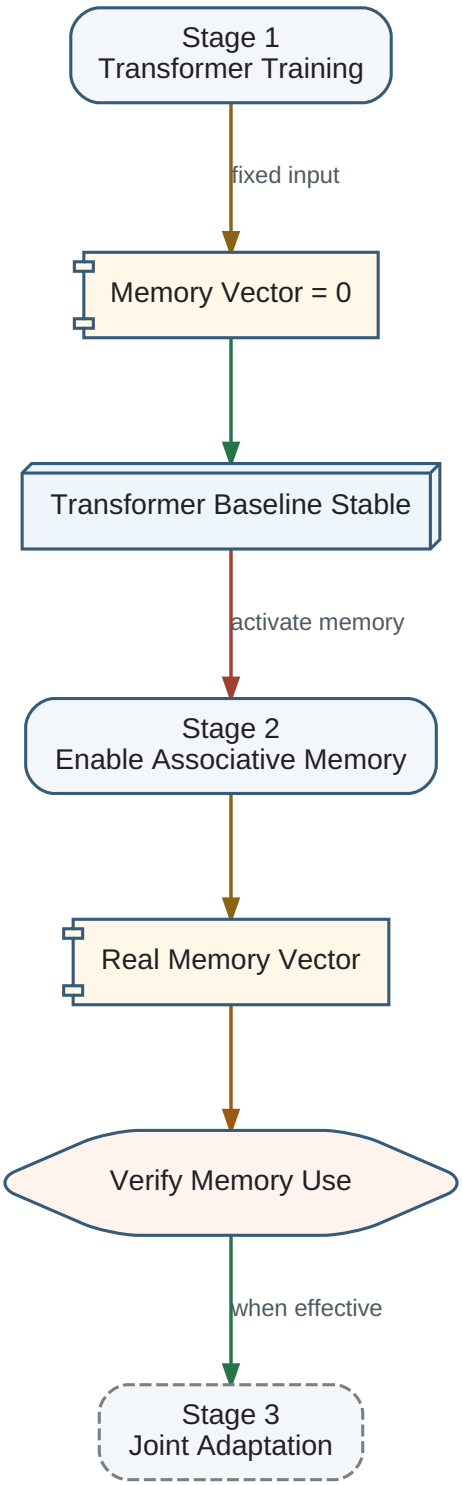
The prototype in this repository implements working memory, long-term memory, functional forgetting, and iterative selection among flat, clustered, and hybrid organizations.

The prototype is not evidence that these three organizations are sufficient. Its purpose is to make the architectural loop executable and measurable.

> **Operational evidence first**
>
> Use the modular prototype to collect failure cases before attempting a deeply integrated memory-model representation.

The Transformer and Associative Memory should be trained in stages rather than optimized as a single coupled system from the beginning. This reduces optimization complexity and gives each subsystem a stable role before the Memory Vector interface is activated.

Progressive Training Strategy



Training begins with a fixed null Memory Vector, then activates the real memory interface, verifies that the Transformer uses it, and finally permits joint adaptation.

Stage 1 — Transformer Training with a Null Memory Vector

During the first stage, the Transformer receives a fixed canonical null Memory Vector, normally the all-zero vector. The Transformer learns language interpretation, reasoning, planning, and answer generation without depending on an immature external memory subsystem.

> ### Null-interface invariant
>
> The dimensionality and placement of the Memory Vector interface should already match the final architecture even while its value is fixed at zero. This prevents a later interface-shape change from becoming entangled with memory integration.

Stage 2 — Associative Memory Integration

After the Transformer baseline has stabilized, Associative Memory is enabled and explicit READ operations begin supplying non-zero canonical Memory Vectors. Training then teaches the Transformer to condition its reasoning on this additional source of knowledge.

> ### Risk of ignoring the Memory Vector
>
> Because the Transformer may already solve many tasks from its parameters and input context, ordinary optimization may converge while the model largely ignores the Memory Vector.

Possible forced utilization

It may therefore be necessary to force Memory Vector utilization during part of training. Candidate mechanisms include examples that cannot be solved without memory, controlled removal of equivalent context, auxiliary losses measuring memory dependence, progressive gating, or temporary constraints on the parameter-only path.

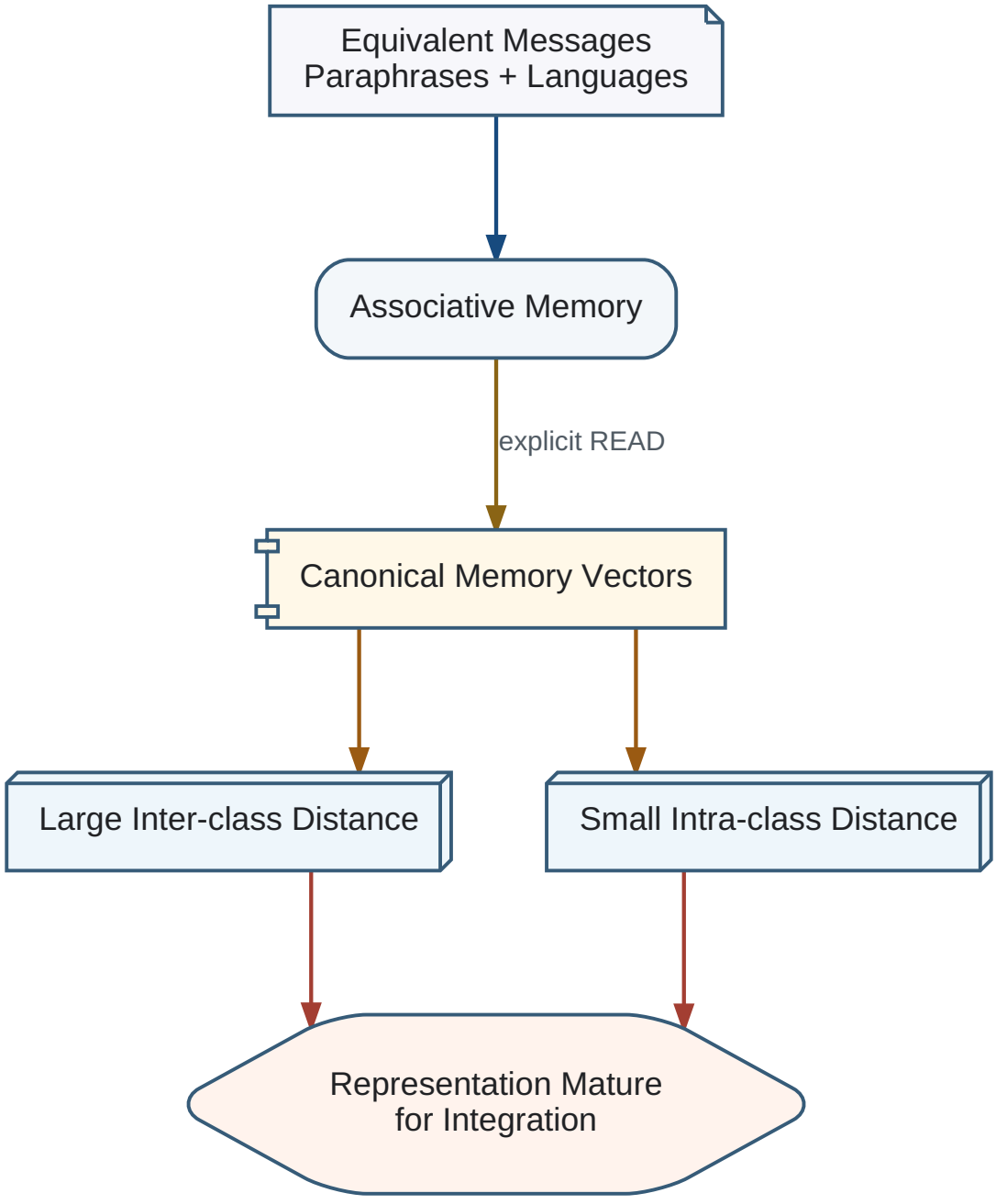
> ### Open engineering question
>
> The specification does not select a single forcing mechanism. The requirement is to measure whether the Memory Vector causally influences the answer and to introduce a forcing mechanism if passive integration is insufficient.

Stage 3 — Joint Adaptation

Joint adaptation should begin only after the Transformer can use the Memory Vector and Associative Memory has reached sufficient representational maturity. The two subsystems remain independently replaceable because the canonical Memory Vector is their stable public interface.

Associative Memory should not be considered ready for Transformer integration merely because its training loss is low. Its representation must become stable with respect to meaning rather than surface form.

Semantic Equivalence Maturity Test



Different formulations and languages expressing the same meaning should converge to nearby canonical Memory Vectors, while different meanings remain separated.

Semantic invariance

Inputs that are differently worded but equal in meaning should generate nearly identical canonical Memory Vectors. This requirement includes paraphrases, different word order, active and passive forms, short and expanded formulations, and equivalent statements in different languages.

```
For Q_i, Q_j in the same semantic equivalence class E:
                d(M(Q_i), M(Q_j)) < epsilon
```

> ### Multilingual equivalence
>
> “The sky is blue.”, “Blue is the color of the sky.”, “Der Himmel ist blau.”, “Небо синее.”, and “Le ciel est bleu.” should produce nearby canonical Memory Vectors after sufficient training.

Semantic discrimination

Semantic invariance must be paired with semantic discrimination. Messages with different meanings must remain separated; otherwise the memory could satisfy invariance by collapsing all inputs to the same vector.

```
For Q_i and Q_k with different meanings:
          d(M(Q_i), M(Q_k)) > delta
```

Operational maturity criterion

Associative Memory is sufficiently mature for integration when intra-class distances remain below an accepted threshold, inter-class separation remains above an accepted threshold, multilingual consistency is stable across checkpoints, and nearest-neighbour retrieval preserves semantic classes.

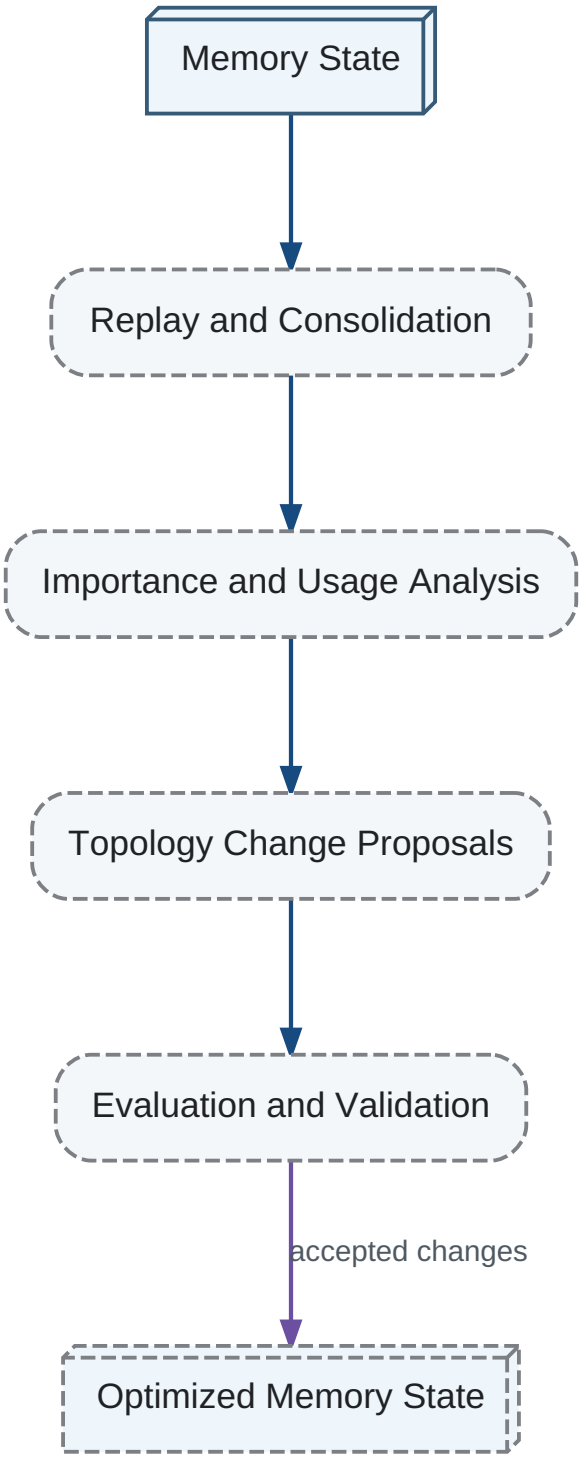
- mean and maximum intra-class distance
- mean and minimum inter-class distance
- multilingual semantic consistency
- nearest-neighbour semantic retrieval accuracy
- stability of the metrics across training checkpoints

Not permanent completion

This criterion marks stabilization of the representation and readiness for Transformer integration. It does not imply that Associative Memory stops learning; Memory State may continue to accumulate knowledge and change over the lifetime of the system.

Local reorganization changes a bounded region of the knowledge graph or memory index. Global reorganization changes cross-domain evaluation, fundamental categories, objective priorities, or the relationship between memory and model.

Offline Sleep and Structural Optimization



A future sleep process replays experience, estimates importance, and proposes safe structural changes to memory.

Local changes should be common. Global changes should require stronger evidence because their errors propagate widely and can alter identity continuity.

Only one major model should normally undergo cardinal restructuring at a time. Rate limits and staged evaluation reduce cascading adaptation to noise. Emergency exceptions remain possible when evidence indicates a global regime change.

> **Conservative structural change**
>
> Adapt quickly through weights and slowly through architecture.

New experience should normally modify the relative importance, usefulness, confidence, and activation probability of existing knowledge before changing architecture.

This is a weaker and more implementable claim than asserting that every important experience reorganizes the model. Weight changes are cheap, reversible in effect, and measurable.

Structural change becomes justified when cumulative reweighting makes the present organization increasingly incoherent or inefficient. No fixed deterministic threshold is assumed.

> ### Locality of ordinary learning
>
> Most daily learning can be represented as local updates. Rare reorganization events should be treated as expensive search episodes.

Sleep-like structural reorganization and the final global evaluation mechanism are intentionally deferred from the first implementation of the Transformer-memory interface.

Deferral is an architectural boundary, not rejection. Initial versions should establish a working memory contract before adding offline topology optimization, forgetting, global reweighting, and more general evaluation.

Neural networks and classical programs both seek states that optimize a task-dependent functional, but they operate under different knowledge conditions.

Class	Functional	Typical executor
I	Well defined and tractable	Algorithm, solver, finite-state machine
II	Not explicit; examples are evaluated	Neural network
III	Partly defined and partly example-based	Hybrid architecture

The third class includes many practical robotics, autonomous driving, diagnosis, planning, and dialogue tasks. The central engineering problem is where to cut the computation.

> **`T_MetaArchitect`**
>
> A trainable system that designs a computational graph by allocating operations among neural, algorithmic, and hybrid executors.

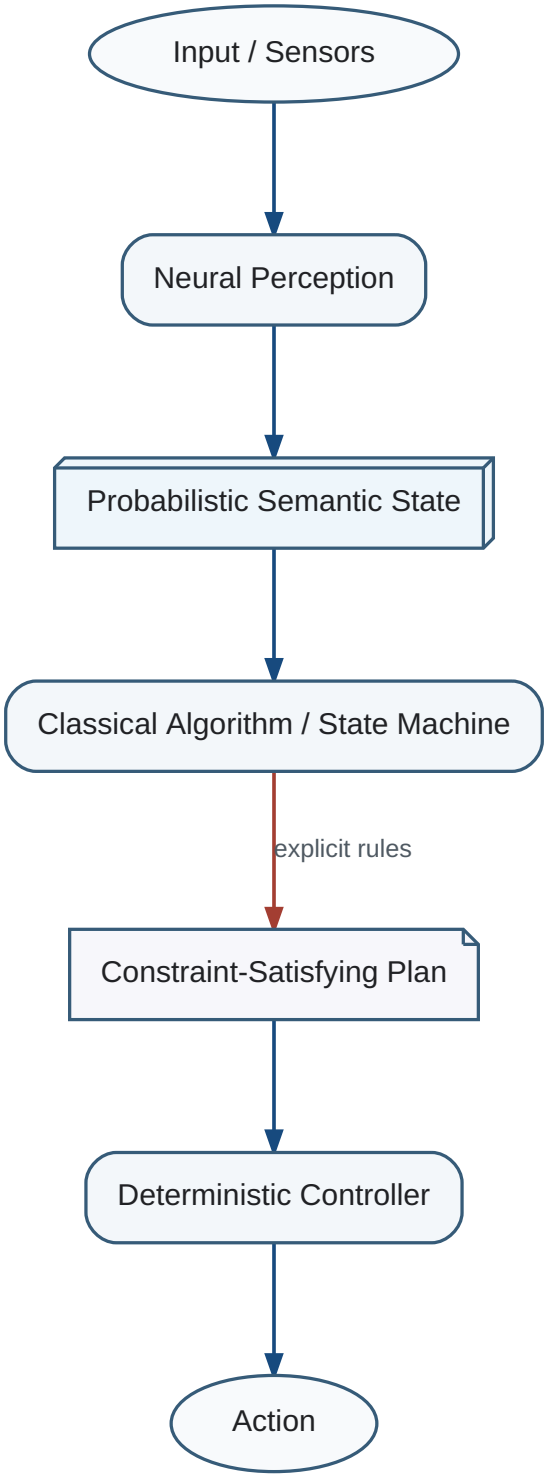
The output of the meta-architect is not the task answer. It is an architecture for producing the answer.

Inputs can include objective completeness, data availability, noise, state discreteness, safety criticality, interpretability requirements, latency, energy cost, and distribution shift.

Observed deployment utility supplies the lifelong training signal.

The optimal unit of allocation is usually not the whole task but an operation or pipeline stage.

Hybrid Neural and Algorithmic Processing



Learned components provide semantic and probabilistic representations; classical algorithms enforce explicit constraints and deterministic control.

```
Sensors -> Neural Perception -> Probabilistic State -> Algorithmic
Constraint Solver -> Planned Trajectory -> Learned Correction ->
                    Deterministic Controller
```

A stage can remain neural where the functional is implicit and data-rich, while safety envelopes, invariants, accounting, and discrete protocol behavior remain explicit.

This division provides interpretability and guarantees where they matter without forcing perception or language into brittle hand-written rules.

As repeated neural solutions reveal stable structure, the system should ask whether part of the behavior can be compiled into a simpler algorithm, rule set, finite-state machine, index, or cache.

Algorithmic compilation reduces cost, improves reproducibility, and exposes invariants. It is a form of crystallized experience.

The transition is not one-way. When the environment changes and exceptions accumulate, an algorithm may become suboptimal and should be returned to probabilistic search.

```
Neural Search <-> Stable Structure <-> Algorithmic Compilation
```

A practical meta-architect can be trained today as a contextual bandit or supervised ranker. Each candidate architecture receives measured accuracy, latency, energy, maintenance cost, interpretability, and failure severity.

The repository implements a lightweight online learner with engineering priors for algorithmic, neural, and hybrid modes. The priors are replaced gradually by outcome data.

The learner should record failures of decomposition separately from failures of the selected executor. Otherwise it cannot distinguish a poor cut from a poor component.

> ### Architecture is itself an optimization target
>
> Do not optimize only the answer. Optimize the computational organization that repeatedly produces answers.

Initial experiments should use tasks with several plausible decompositions and measurable non-accuracy costs. Autonomous-control simulations, document processing, tool-using agents, and software maintenance are suitable candidates.

Baselines include all-neural, all-algorithmic, fixed hybrid, human-designed hybrid, and learned hybrid architectures.

The highest-value result may be an allocator that improves only modestly on current tasks but creates a reusable architecture-search loop for future systems.

> ### Branching value
>
> An improvement that opens many later architectures can be more valuable than a larger immediate benchmark gain that leaves the system structurally unchanged.

Written text, speech, lectures, and dialogue all provide linguistic information. Their channel properties differ, but each can trigger `T_Deserialization` and `T_ModelIntegration`.

Speech adds timing, emphasis, and emotional coloration. Written text permits arbitrary pacing, exact revisitation, annotation, and repeated integration attempts.

A lecture combines prepared serialization with real-time adaptation to questions. Dialogue adds an iterative feedback loop in which the sender can change later serialization based on observed integration failure.

A significant book is not merely a container of propositions. It is a designed sequence of stimuli capable of changing categories, evaluations, attention, and future questions.

The reader may forget most explicit sentences while retaining a changed system of integrated evaluation. The durable output is therefore a modified reader, not a copied text.

> ### Cognitive value of a book
>
> The measurable change in future interpretation, evaluation, learning direction, and action induced by iterative integration of the book.

This definition permits evaluation without assuming perfect recall.

The same text produces different integrations as the reader changes. A book read at twenty, forty, and sixty can be three different cognitive events because the background model and objectives differ.

An artificial cognitive system should revisit important books after substantial model change. Repetition should continue while rereading materially changes the model or the direction of future learning.

A stronger stopping rule is not merely no further factual change. Reading can remain valuable while changing which questions receive attention. Iteration stops when additional exposure no longer changes either the model or the learning trajectory within the relevant objective class.

```
Read -> Integrate -> Act and Learn -> Changed Model -> Re-read
```

A mature distributed cognitive state cannot generally be copied into another independently evolved state. The same experience derives meaning from system-wide relations absent in the receiver.

Dialogue is therefore not an inferior substitute for Merge. It may be the maximum viable transfer operation for integrated cognitive systems.

The sender serializes selected structures; the receiver integrates them; clarification reveals missing background; conflict reveals incompatible organization; both systems update through the process.

> ### Dialogue maximum
>
> For mature non-identical integrated models, no general operation transfers experience more completely than iterative dialogue without replacing the recipient's identity.
>
> Confidence: 0.86

An expert cannot transfer neural state directly. Education instead designs a sequence of observations, tasks, failures, explanations, and feedback intended to move another system through a productive learning trajectory.

Books, curricula, and apprenticeships are therefore programs for inducing self-generated internal changes.

Future cognitive systems may exchange not memories but training trajectories: ordered experiences selected because they produced useful restructuring in the source system.

> ### Culture as external memory
>
> Culture stores serializations and learning trajectories that can reshape many independent cognitive systems without merging them.

A reading experiment should measure model change before and after a text, behavior on later unrelated tasks, attention allocation, question generation, and the value of rereading after intervening experience.

A dialogue experiment should compare one-way transfer, clarification-enabled transfer, and full iterative model conflict resolution.

The repository's communication prototype supplies the minimal serialization, deserialization, missing-background, integration, and conflict objects required for such experiments.

> ### Measure transformation, not recall alone
>
> Immediate factual recall is one output of linguistic integration, not the primary measure of cognitive change.

Digital systems can clone a high-performing model at negligible marginal cost. Global cloning maximizes immediate exploitation but reduces diversity and makes the population vulnerable to unforeseen environmental changes or shared failure modes.

Individual evolution without cloning preserves diversity but sacrifices commercial efficiency and reproducibility.

The proposed compromise separates selection within lineages from selection across lineages.

> **`T_CognitiveLineage`**
>
> A bounded class of cognitive systems sharing architecture, objective structure, and inherited state while remaining distinct from other lineages.

A population maintains a small number of strongly differentiated lineages. Inside a lineage, the best current system can be cloned for operational deployment. Across lineages, no single champion replaces all alternatives.

Lineages may specialize in scientific reasoning, social modeling, physical control, creative exploration, adversarial robustness, or other objective classes.

The number of lineages should be small enough for commercial maintenance and large enough to preserve meaningful diversity.

Ants, termites, and bees demonstrate a successful biological compromise: large numbers of similar or genetically close agents operate efficiently inside a colony while colonies and species preserve population-level variation.

The analogy is not exact. Digital cognitive lineages can share tools, Research State, and selected learning trajectories at much higher bandwidth. The useful principle is multilevel selection rather than literal imitation of insect biology.

> ### Two optimization levels
>
> Exploit the current champion inside each lineage; preserve diversity and competition among lineages.

Direct internal-state Merge between mature lineages is expected to be destructive or undefined. Exchange should use dialogue, externalized Research State, experiments, books, and training trajectories.

This protects identity and diversity while permitting useful discoveries to propagate.

A receiving lineage integrates exported knowledge according to its own objectives. It need not reproduce the source representation or conclusions exactly.

```
Internal Model A -> Partial Serialization -> Research State /
      Dialogue -> Model Integration -> Internal Model B
```

A provider can operate several maintained lineage families and deploy many cloned instances of the best current version within each family.

Customers select a lineage by objective class and risk profile rather than receiving one universal model. Updates can be tested within a lineage before promotion.

Failure in one lineage does not automatically corrupt all deployed cognition. Diversity becomes an operational resilience mechanism, not an academic luxury.

The repository includes a lineage manager that clones champions within lineages and measures distance among lineage signatures.

Lineage specialization can produce incompatible values, communication failures, and hidden shared dependencies. Apparent diversity may be superficial when all lineages inherit one base model or evaluator.

Evaluation must therefore distinguish architectural diversity, training-data diversity, objective diversity, and operational independence.

Cross-lineage review and adversarial testing should be institutionalized without forcing convergence.

> ### Preserve meaningful variance
>
> Do not measure diversity only by parameter distance; measure different failure modes, objectives, abstractions, and responses to environmental change.

> **`T_ResearchState`**
>
> The canonical implementation-independent state of the research model from which human-readable and machine-readable artifacts are generated.

Research State stores the current consolidated model, not the full dialogue transcript. It includes Tokens, principles, hypotheses, architecture decisions, open questions, roadmap items, chapter structure, and the latest consolidation record.

The state must support recovery after session loss, use by a different model, and long-term local ownership.

Documentation is a partial serialization of Research State for a human audience. English and future language versions are separate serializations, not an original plus literal translations.

Every formal Token uses the case-sensitive form `T_PascalCaseIdentifier`.

Only Atomic Tokens may have natural-language definitions. Every non-atomic Token is defined through previously declared Atomic Tokens and formal composition operators.

Atomic Tokens act as axioms. A mature language should reduce their number while preserving or increasing expressive power.

Token semantic identity is immutable. A definition may be refined or expressed more formally, but semantic replacement requires a new Token. Deprecated Tokens remain available for compatibility.

```
Language Maturity ∝ Expressive Power / Atomic Token Count
```

YAML is used as a readable serialization of the formal state. Its flexibility is restricted by schema and profile rules.

- Mappings with string keys
- Sequences
- Strings, booleans, finite numbers, and null
- No anchors or aliases
- No merge keys or custom tags
- No duplicate keys
- No implicit date or time values

Canonical formatting produces stable field order and entity order while preserving semantically ordered block sequences.

The format remains parseable by standard YAML tools and is validated against JSON Schema plus project-specific semantic rules.

Git is the sole history mechanism. Research State stores current state, not a cumulative internal history.

Each version contains a latest `T_ConsolidationRecord` describing only the semantic transition that produced that version. Earlier records remain available in earlier Git commits.

When a commit contains no semantic change, the consolidation record may remain unchanged. File diffs then record formatting or generated-artifact changes without inventing a false knowledge transition.

Three-way file merge is structural. True semantic consolidation can produce a result belonging to neither branch. The tool therefore emits conflict reports for review rather than pretending every textual merge is a cognitive merge.

HTML is never the canonical source. The generator reads chapter structure and semantic content blocks from Research State.

Each page contains a single-line navigation bar with Up, Previous, Contents, A–Z Index, and Next links at both the top and bottom. Large chapters use a directory index and section files; small chapters can remain a single file.

Semantic CSS classes identify Tokens, definitions, hypotheses, observations, examples, notes, warnings, formulas, and principles. Presentation can change without editing content.

The current stylesheet follows a restrained ISO, RFC, and W3C-inspired layout with a fixed reading width and a small left indent for body text.

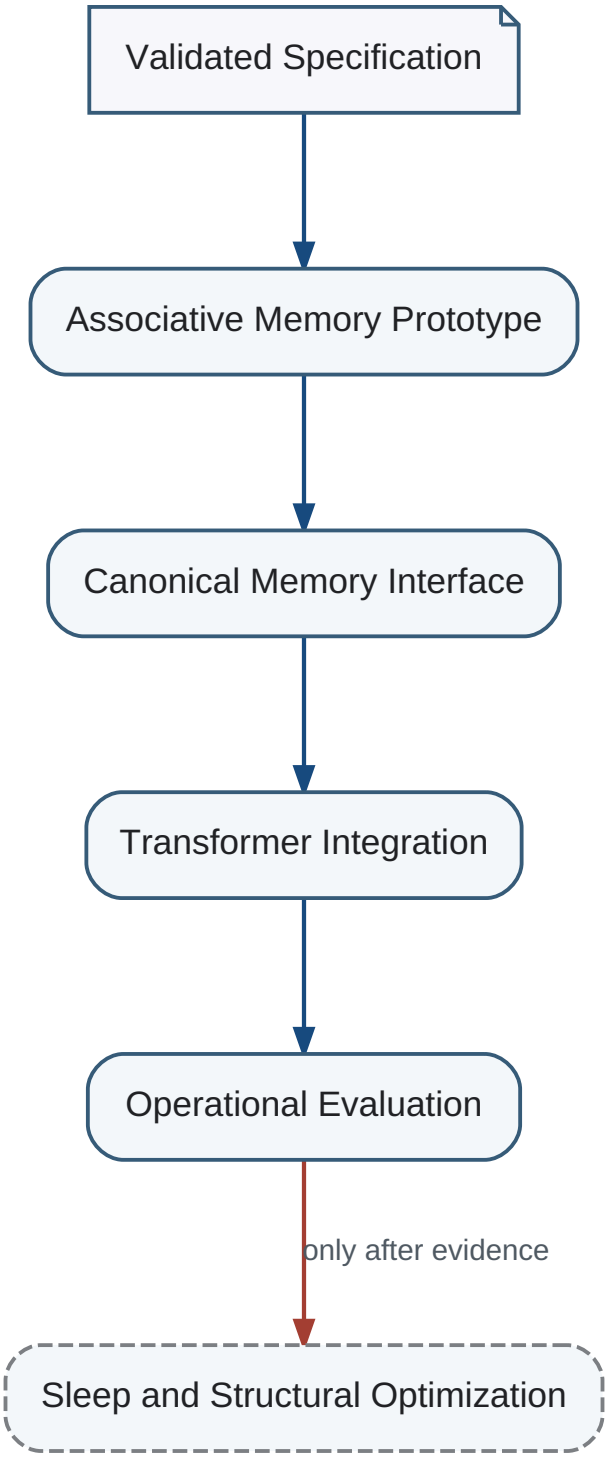
The initial compiler and schema are written in ordinary Python, JSON Schema, and YAML. As the language matures, its own schema, consolidation rules, and generator descriptions can be represented inside Research State.

This is analogous to rewriting a compiler in the language it compiles. Self-hosting is a maturity target, not a requirement for the first operational version.

The research environment then becomes an experiment in the architecture it describes: working discussion, consolidation, long-term state, partial serialization, review, and further integration.

> ### Use the project to build the project
>
> A mechanism should first prove useful as research infrastructure before becoming a candidate component of the cognitive architecture itself.

Implementation Sequence



The roadmap introduces the smallest useful architecture first and defers topology optimization and global maintenance until evidence justifies them.

- Restricted YAML Research State
- Atomic and derived Token validation
- Canonical formatting
- Semantic change proposals
- Structural three-way merge with conflict report
- Generated semantic HTML
- Working and long-term memory prototype
- Lossy consolidation and architecture selection
- Communication and clarification pipeline
- Trainable multichannel evaluator
- Hybrid meta-architect
- Cognitive lineage manager

These components are deliberately modular. Their purpose is to create an operational baseline and instrumentation for later integration.

The highest-priority experiment is a persistent agent operating for months with exact external logs and lossy internal memory. The experiment should measure retention, adaptation, interference, objective drift, and recovery from false consolidation.

A second experiment trains the meta-architect on multiple task families and compares learned decomposition with fixed all-neural, all-algorithmic, and human-designed hybrid baselines.

A third experiment measures cognitive transformation from books and dialogue using future task behavior and question generation rather than recall alone.

A fourth experiment maintains several lineages under simulated environmental regime changes and compares global champion cloning with lineage-preserving deployment.

Only after modular experiments produce stable evidence should the project attempt a shared latent state integrating memory, model, and evaluation.

The integrated system must be evaluated on continuous learning, objective drift, communication, identity continuity, correction, and external auditability.

Failure should be informative. If integrated memory is unstable, the modular Research State and exact logs allow reconstruction of what changed and why.

The current Research State contains formal open-question entities. The most consequential unresolved areas are listed below.

- Minimal Atomic Token basis
- Integrated memory-model representation
- Architectural-search trigger and candidate generation
- Stable lifelong training of global evaluation
- Optimal neural-algorithmic split features
- Identity continuity across hardware migration
- Population resilience and lineage count
- Boundary between emergent and explicitly required higher-level behavior

The open questions define the research program. They are not defects to hide and should not be replaced by confident prose before experiments exist.

The project does not have a fixed final architecture. A release is successful when it improves the current working system, preserves state recovery, documents semantic change, and makes the next experiment easier to perform.

A mature long-lived cognitive system would preserve continuity while changing almost every internal component over time. The research program should exhibit the same property.

> ### Evolutionary completion
>
> Do not optimize for a final immutable design. Optimize for an architecture that can keep improving without losing accumulated knowledge or the ability to explain its own development.

This chapter defines a prospective hardware realization path. It does not replace the technology-independent cognitive architecture and must preserve all canonical invariants.

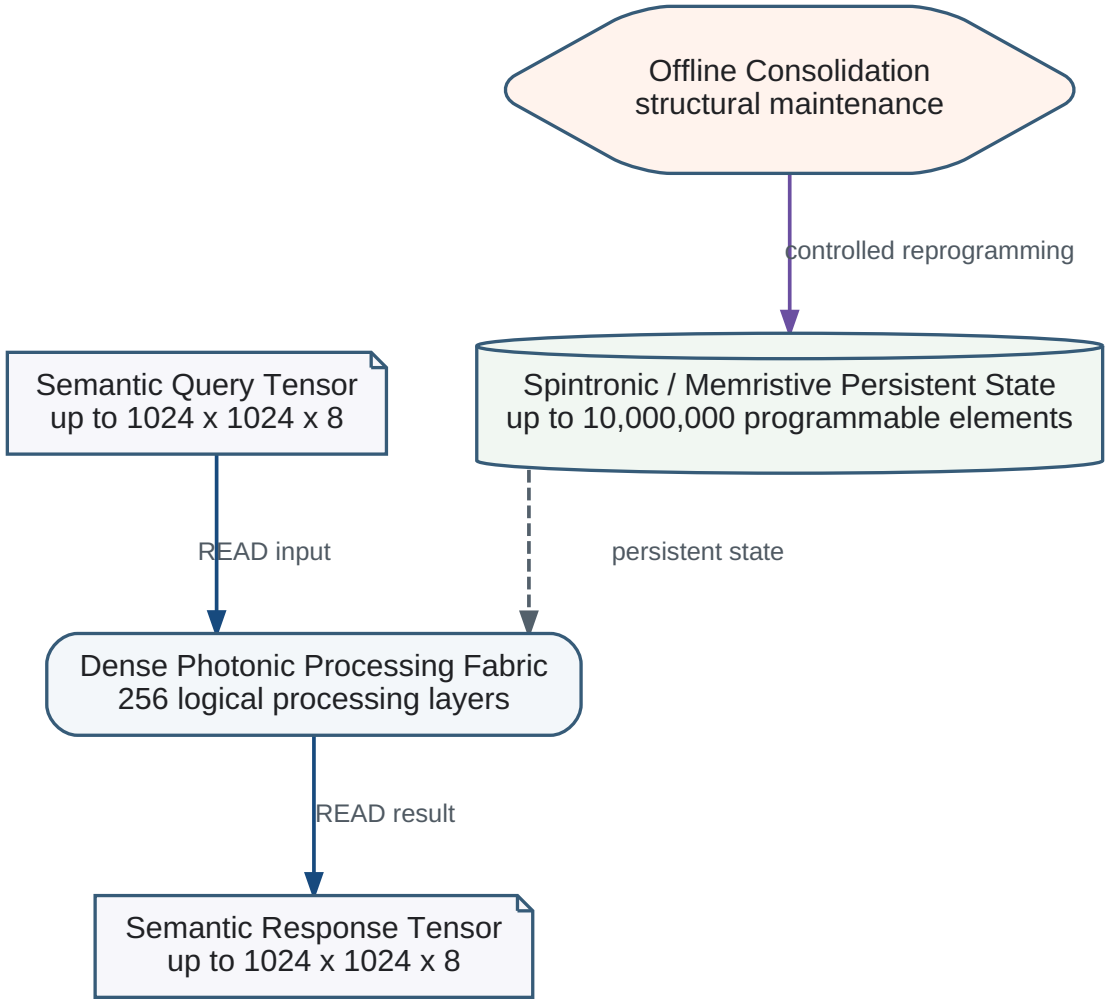
> **Technology-neutral core**
>
> Associative Memory, READ, UPDATE, Memory State, Memory Vector, Offline Consolidation, and semantic basis maintenance remain logical contracts independent of a specific material platform.

The preferred research direction is a distributed dense photonic-spintronic associative tensor memory combining photonic transport and transformation with nonvolatile programmable material state.

Functional Partition

- Photonic fabric distributes and transforms semantic tensors at high bandwidth.
- Spintronic, memristive, or phase-change elements store persistent coefficients and programmable state.
- A dense layered processing fabric applies semantic transformations.
- Offline Consolidation performs controlled structural maintenance and hardware reprogramming.

Distributed Photonic-Spintronic Associative Tensor Memory



A prospective dense hardware implementation preserving explicit READ and UPDATE separation.

Photonic and material subsystems may be tiled and distributed. Distribution is a physical scaling strategy; it does not imply a sparse semantic architecture.

Scale Parameters

Semantic query and response representations may reach 1024 x 1024 x 8 elements, or 8,388,608 elements per tensor. Processing depth is an experimental implementation parameter ranging from 8 to 256 layers. The value 256 is an upper experimental bound and must not be interpreted as query, response, or Memory Vector dimensionality.

The architectural scale parameter 10,000,000 x 256 denotes a dense logical organization of up to ten million programmable processing or memory positions considered across an experimental depth of 8 to 256 processing layers. It is not a table of ten million 256-dimensional stored vectors.

Representation	Elements	INT4	INT8	INT16
One 1024 x 1024 x 8 tensor	8,388,608	4 MiB	8 MiB	16 MiB
Input plus output	16,777,216	8 MiB	16 MiB	32 MiB
Upper-bound 256 full intermediate tensors	2,147,483,648	1 GiB	2 GiB	4 GiB

Processing Depth

The architecture does not prescribe a fixed processing depth. The reference implementation supports an experimental range of 8 to 256 layers. The required depth must be determined experimentally from associative learning quality, retrieval accuracy, and computational cost. A released implementation fixes the selected depth for that release.

A physical implementation may use pipelining, temporal reuse, tiling, wavelength multiplexing, spatial multiplexing, polarization, or modal channels. These are implementation strategies for a dense logical architecture.

Dense Topology and Basis Compaction

The implementation does not depend on sparse candidate routing as its primary model. Sequential localized orthogonalization removes semantic redundancy, transfers correlated magnitude into retained coordinates, and progressively compacts useful information.

Dense hardware invariant

Temporary inactive coordinates may reduce activity and power, but all released coordinates remain addressable until validated dimensional reduction is performed in a later release.

Precision Tiers

- INT4 provides maximum storage density and bandwidth efficiency.
- INT8 is the default operational precision.
- INT16 is reserved for sensitive layers, accumulation, calibration, high-dynamic-range state, and Offline Consolidation statistics.

INT8 may be implemented through two INT4 slices, and INT16 through multiple lower-precision slices. Bit-sliced realization is preferred when a single physical cell cannot provide sufficiently stable multilevel precision.

Physical READ / UPDATE Separation

- READ uses non-destructive sensing paths and never applies programming pulses.
- UPDATE uses dedicated programming paths and never performs retrieval.
- UPDATE modifies persistent state but never produces a Memory Vector.
- A new Memory Vector is produced only by an explicit later READ.

Offline Consolidation

Offline Consolidation may recalibrate devices, replay important states, strengthen or weaken associations, maintain the semantic basis, collect dimensional-usage statistics, and produce release-level architectural recommendations. It must not change runtime dimensionality.

> ### Research status
>
> Photonic-spin glass, magnonic, spintronic, memristive, and phase-change implementations remain prospective technologies. This chapter defines architectural compatibility and evaluation criteria rather than claiming an existing complete implementation.

Part V documents the intended evolution of Cognitive beyond release 0.3.15. It is an architectural roadmap rather than a promise that every mechanism is already implemented.

Preserve canonical invariants

Future mechanisms must preserve the explicit READ/UPDATE asymmetry and the rule that Associative Memory learns exclusively from Transformer-produced Semantic Representations through the Semantic Feedback Learning Pipeline.

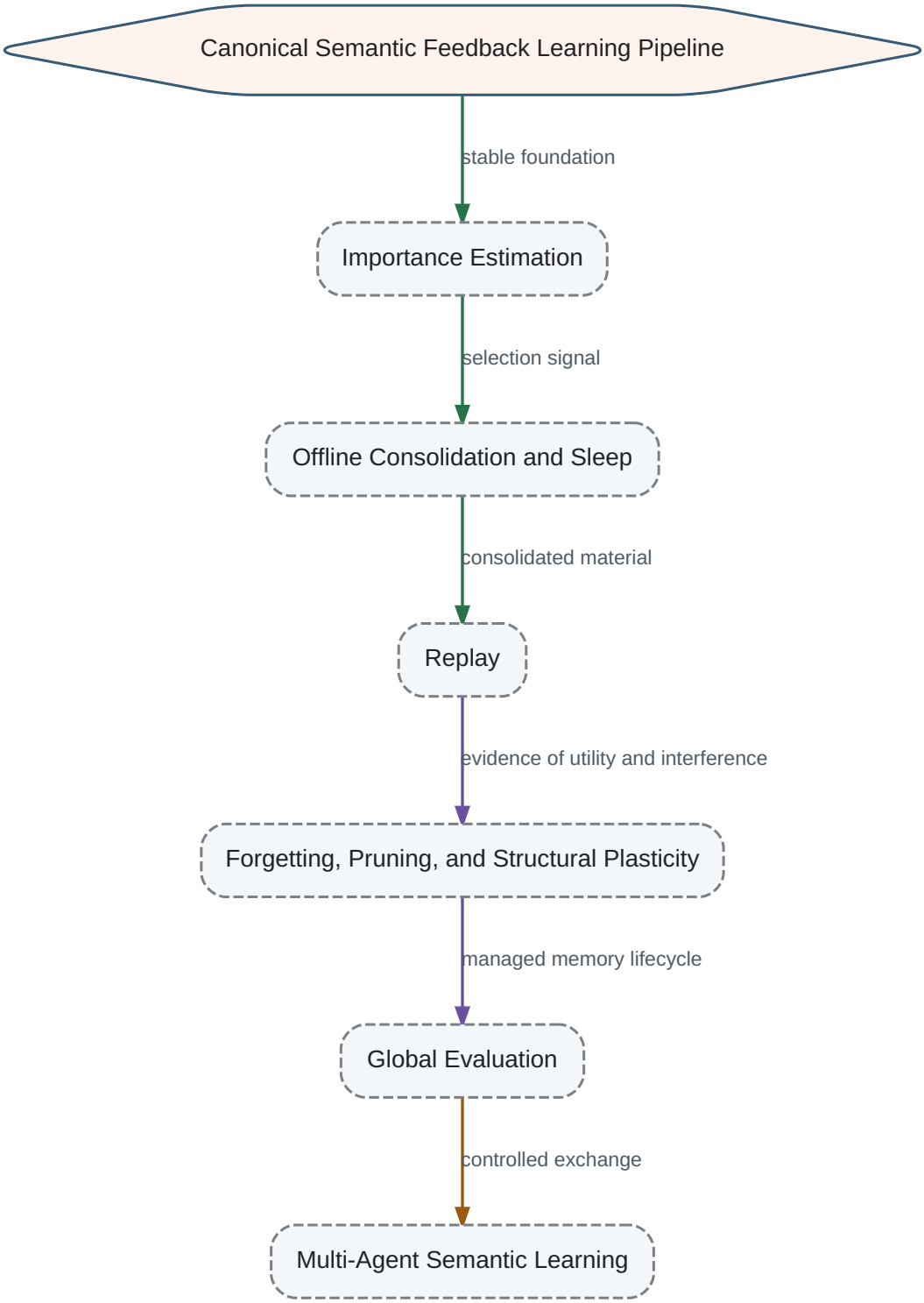
- Extend the architecture through explicit subsystems rather than hidden side effects.
- Separate reasoning, evaluation, learning, and structural maintenance responsibilities.
- Introduce mechanisms only when their contracts, state effects, and failure modes can be stated clearly.
- Keep canonical information in one source and generate derivative documentation from it.
- Maintain human readability so engineers can reason about the system without unnecessary formalism.

Roadmap status

The order in this part expresses architectural priority and dependency. It does not imply that every mechanism forms one rigid runtime pipeline.

The next development cycle begins with the ability to estimate the importance of semantic knowledge. Consolidation requires a selection signal; replay requires a policy for deciding what should be reactivated and why.

Canonical Development Sequence



The canonical Semantic Feedback Learning Pipeline is the stable foundation. Importance Estimation, Offline Consolidation and Sleep, and Replay are introduced as explicit dependent subsystems; later memory-lifecycle and evaluation mechanisms build on them.

- 1. Importance Estimation
- 2. Offline Consolidation and Sleep
- 3. Replay
- 4. Forgetting and Memory Pruning
- 5. Structural Plasticity
- 6. Global Evaluation
- 7. Multi-Agent Semantic Learning

Importance Estimation

A future subsystem that assigns context-sensitive significance to Semantic Representations, memory structures, associations, and experiences.

Importance is required before the system can decide what deserves retention, consolidation, replay, protection from forgetting, or structural investment.

- Estimate immediate task relevance.
- Estimate expected long-term utility.
- Record novelty, recurrence, confidence, emotional or evaluative significance, and causal relevance where applicable.
- Distinguish frequently accessed knowledge from knowledge that is merely duplicated.
- Permit importance to change as goals, context, and later evidence change.

No direct raw-observation scoring

Importance Estimation operates on semantic objects and system context. It must not create a second path through which raw external observations directly train Associative Memory.

Relationship to Integrated Evaluation

Importance Estimation may consume evaluative signals, but it is not identical to the future Global Evaluation subsystem. Importance concerns memory lifecycle decisions; Global Evaluation concerns system-wide significance and action selection.

Offline Consolidation

A future process that reorganizes Memory State outside the immediate online response path, using accumulated semantic knowledge and importance evidence.

Sleep

The operating mode in which offline consolidation, topology analysis, compression, pruning, and structural learning may occur with reduced interference from online activity.

Sleep is not defined as passive inactivity. It is a controlled architectural mode for expensive memory maintenance that should not be mixed invisibly with ordinary READ or UPDATE operations.

- Group semantically related knowledge.
- Resolve redundancy and selected contradictions.
- Compress repeated semantic structures.
- Promote stable knowledge and weaken low-value structures.
- Collect evidence for structural changes without requiring them during every online interaction.
- Preserve rollback and provenance so destructive consolidation can be detected and reversed.

Semantic boundary

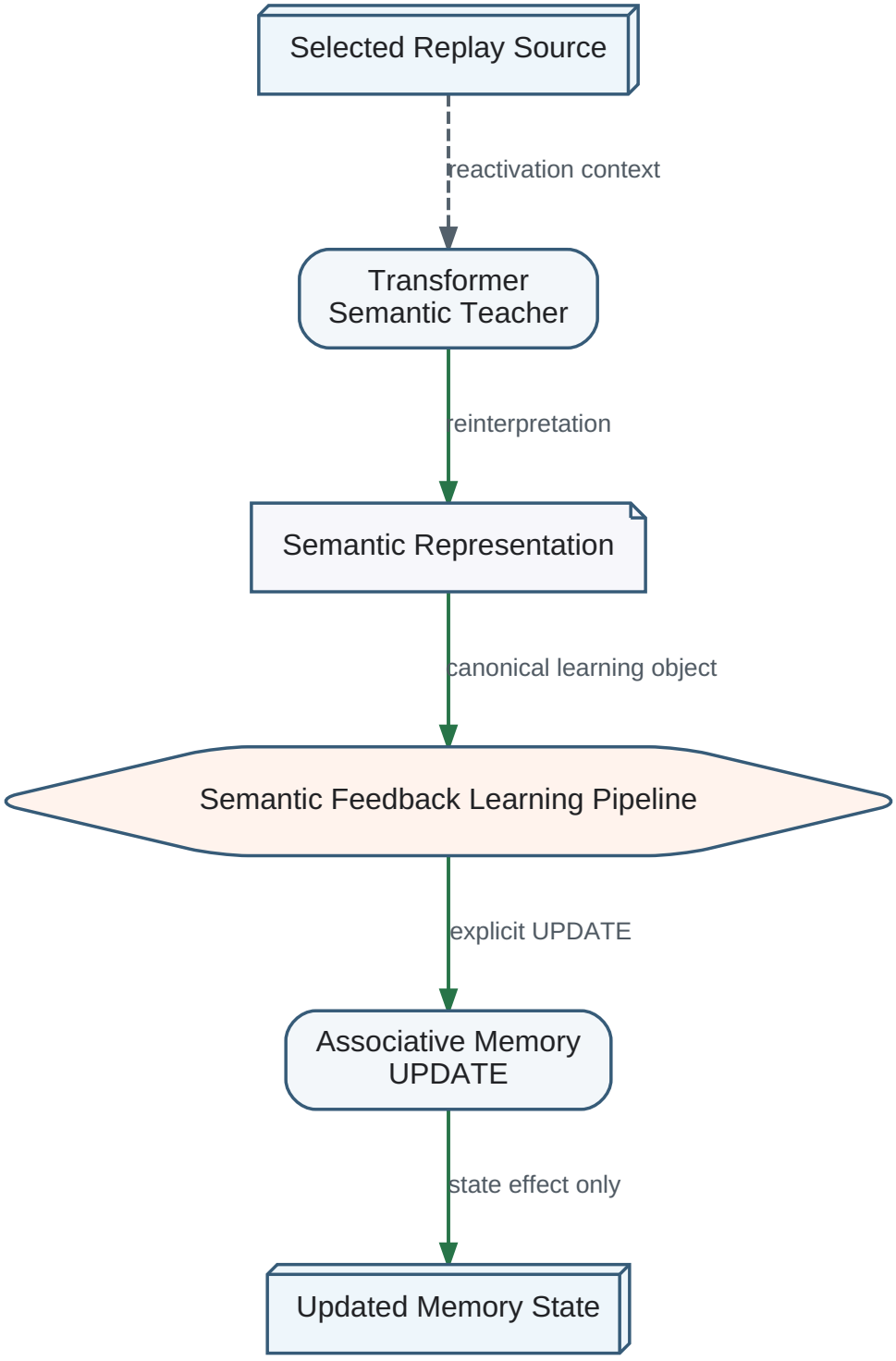
Offline mechanisms may inspect Memory State and Transformer-produced Semantic Representations, but any ordinary learning update must continue to obey the canonical Semantic Feedback Learning contract.

> **Replay**
>
> Controlled reactivation of selected semantic experience so the system can verify, reinforce, compare, reinterpret, or intentionally weaken memory structures.

Replay follows Importance Estimation and the initial design of Offline Consolidation because it requires both a selection policy and a protected execution context.

- Priority replay for high-importance or unstable knowledge.
- Diversity replay to prevent narrow recent experience from dominating memory.
- Counterexample replay to test overgeneralized associations.
- Recovery replay after rollback or detected false consolidation.
- Cross-context replay to measure whether knowledge remains useful under changed goals or environments.

Canonical Replay Boundary



Replay reactivates semantic material through the Transformer. It does not bypass the Semantic Feedback Learning Pipeline or write raw stored observations directly into Associative Memory.

Explicit retrieval remains mandatory

When replay needs current memory context, it must perform an explicit READ. UPDATE never performs implicit READ and never produces a Memory Vector.

A mature memory system requires an explicit lifecycle rather than unlimited accumulation.

Forgetting

Controlled reduction of accessibility, precision, or persistence for knowledge whose expected utility no longer justifies its cost or interference risk.

Memory Pruning

Removal or compression of redundant, obsolete, low-confidence, or harmful structures according to validated policy.

Structural Plasticity

Modification of the topology of Associative Memory, including creation, strengthening, weakening, merging, separation, or removal of associations.

- Forgetting must be measurable and reversible where practical.
- Pruning decisions should use importance, confidence, usage, redundancy, and interference evidence.
- Structural learning must remain distinct from functional online learning.
- Connection-level state may accumulate evidence for later topology changes.
- No mechanism may silently violate persistent-memory integrity or provenance requirements.

Global Evaluation

A future conservative subsystem that integrates goals, Memory State, context, predicted consequences, and candidate actions into a final system-level evaluation.

Global Evaluation is deferred because it influences every major behavior and must be introduced only after its authority, training path, stability, and governance are explicit.

Multi-Agent Semantic Learning

Exchange of Transformer-produced Semantic Representations between Cognitive systems without copying entire private Memory States as the default learning mechanism.

- Evaluate imported representations before UPDATE.
- Preserve source identity, confidence, provenance, and version.
- Detect semantic disagreement instead of treating transmission as guaranteed understanding.
- Permit lineage diversity and prevent a single erroneous agent from globally overwriting population memory.
- Use explicit policies for trust, quarantine, reconciliation, and rollback.

Semantic diversity and the Babel hypothesis

The communication-level argument for preserving partially independent semantic lineages is developed in Section 3.6, Higher-Level Communication Phenomena. Part V adopts that discussion as a design consideration for Multi-Agent Semantic Learning rather than duplicating it here.

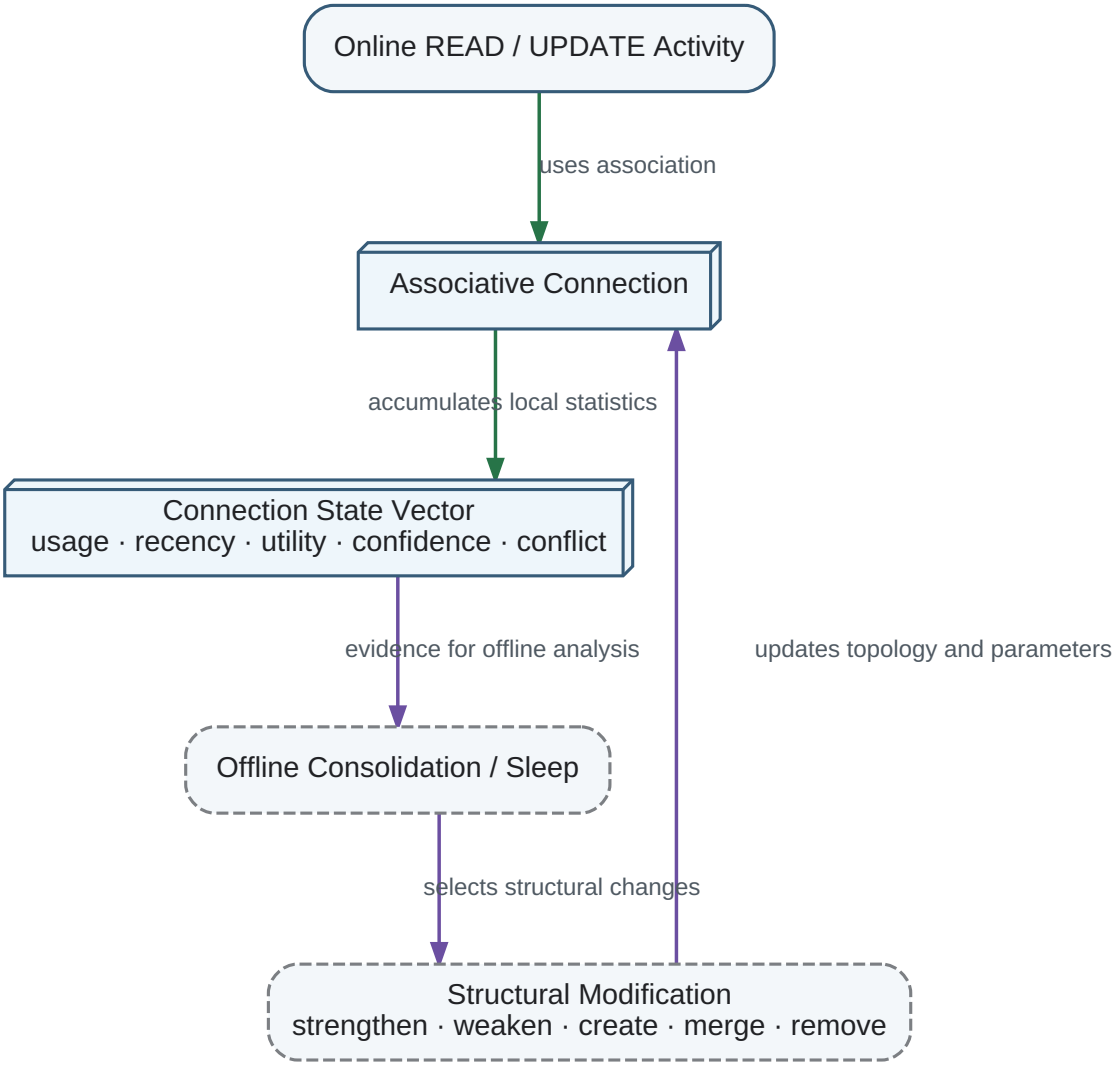
Exchange boundary

Systems exchange semantic representations or other explicit messages. Receiving Associative Memory still learns only through its local Semantic Feedback Learning Pipeline.

> **Connection State Vector**
>
> A persistent state vector attached to each associative connection. It accumulates local evidence about how the connection is used, including activation frequency, recency, co-activation context, semantic utility, confidence, conflict, and contribution to successful reconstruction.

Ordinary online operation updates connection-state statistics without immediately changing memory topology. This separates inexpensive functional learning from expensive structural learning.

Connection-State Evidence and Sleep-Time Structural Learning



Online activity accumulates local statistics in each associative connection. Offline Consolidation and Sleep later use that evidence to modify topology, connection strength, routing, merging, creation, or removal.

Neuromodulator analogy

The Connection State Vector is architecturally analogous to locally accumulated biochemical modulation around biological synapses: repeated use, salience, reward, stress, novelty, and other context-dependent signals can leave traces that later influence plasticity. This is an engineering analogy, not a claim that the vector literally reproduces any specific neurotransmitter or neuromodulator mechanism.

- Online phase: accumulate statistics and permit small reversible parameter updates.
- Sleep phase: compare evidence across many connections and episodes.
- Structural phase: strengthen, weaken, create, merge, reroute, or remove connections.
- Preserve provenance, rollback information, and uncertainty for every destructive change.

Functional and structural learning remain separate

Connection State Vectors collect evidence during normal operation. They do not authorize immediate topology changes. Expensive structural modification is performed only by an explicit Offline Consolidation / Sleep subsystem under validation and rollback controls.

The Research Agenda converts architectural direction into experiments that can falsify assumptions. A future subsystem is not accepted merely because it is biologically plausible or intuitively attractive.

- State a hypothesis and competing alternatives.
- Define observable measurements before implementation.
- Use persistent agents and longitudinal experiments where memory effects require time.
- Record external ground-truth logs separately from lossy internal memory.
- Measure failure, recovery, and unintended interactions as well as average performance.
- Promote a mechanism from research to architecture only after its contract and evidence are reviewable.

Importance Estimation should be studied before consolidation and replay policies are fixed.

Experiment	Primary measurement	Failure signal
Retention prediction	Correlation between estimated importance and future task utility	High-value knowledge is forgotten while low-value noise is preserved
Context shift	Ability to revise importance after goals change	Scores remain frozen despite changed utility
Novelty versus frequency	Separation of rare critical knowledge from frequent trivial knowledge	Frequency dominates all other signals
Interference prediction	Ability to identify knowledge likely to cause destructive overlap	Unexpected degradation after UPDATE or consolidation

> **Acceptance direction**
>
> No single scalar is assumed to be sufficient. The representation of importance remains an open design choice until experiments justify it.

Offline Consolidation and Sleep experiments should compare online-only learning against scheduled maintenance under identical experience streams.

- Measure retention, compression, retrieval quality, interference, contradiction handling, and recovery from false consolidation.
- Compare fixed schedules, activity-triggered schedules, and resource-triggered schedules.
- Test whether offline reorganization improves later READ results without changing the READ contract.
- Measure the cost of consolidation and the effect of interrupted or incomplete sleep cycles.
- Require snapshot, rollback, and audit trails for destructive operations.

Replay experiments must separate the benefits of reactivation from simple repeated exposure.

- Compare importance-prioritized, recency-based, uniform, diversity-preserving, and counterexample replay.
- Measure catastrophic forgetting, overfitting to recent experience, semantic drift, and recovery speed.
- Test replay under changed goals to determine whether old knowledge is reinterpreted or merely reinforced.
- Measure whether replay increases false confidence or amplifies corrupted memories.
- Verify that all learning effects occur through explicit Semantic Representations and UPDATE.

Memory lifecycle experiments combine importance, consolidation, replay, forgetting, pruning, and structural plasticity over long operating periods.

- Run agents for months or equivalent accelerated trajectories.
- Maintain exact external event logs for retrospective comparison.
- Track memory size, retrieval latency, semantic coverage, contradiction density, and task performance.
- Introduce controlled misinformation and measure quarantine, correction, and residual contamination.
- Compare reversible weakening with irreversible deletion.
- Test topology changes against stable non-structural baselines.

Global Evaluation and multi-agent experiments require stronger governance because errors can influence both behavior and other systems.

- Compare local evaluation with proposed global evaluation on long-horizon decisions.
- Measure objective drift and recovery after misleading feedback.
- Exchange semantically equivalent messages across languages and formulations and measure reconstruction consistency.
- Test disagreement detection, trust calibration, quarantine, and rollback across lineages.
- Prevent benchmark success from being mistaken for general safety or cognitive maturity.

The long-term target is a cognitive platform that can reason semantically, maintain a persistent and evolving world model, learn continuously, and improve its own memory organization without collapsing reasoning, learning, and evaluation into one opaque operation.

> ### Evolution without architectural amnesia
>
> Future releases should extend the stable abstractions of earlier releases or explicitly document why an invariant must be replaced. Silent reversion to direct observation-driven Associative Memory learning is prohibited.

Memory should evolve from a passive store into a managed lifecycle with explicit acquisition, retrieval, importance, consolidation, replay, weakening, pruning, and structural adaptation.

- Persistent but not immutable.
- Lossy but auditable.
- Adaptive but bounded by contracts.
- Capable of forgetting without losing provenance of the forgetting decision.
- Capable of structural change without confusing topology optimization with ordinary semantic learning.

Cognitive capabilities should appear as explicit architectural subsystems with named inputs, outputs, state effects, and ownership.

- Semantic Reasoning remains distinct from memory maintenance.
- Semantic Feedback Learning remains the ordinary learning boundary.
- Importance Estimation governs lifecycle priority.
- Offline Consolidation and Sleep provide protected maintenance.
- Replay provides controlled reactivation.
- Global Evaluation provides conservative system-wide significance when it becomes sufficiently specified.

Multiple Cognitive systems may eventually form populations that exchange experience while preserving distinct Memory States and cognitive lineages.

- Share Semantic Representations rather than opaque memory copies by default.
- Preserve diversity to reduce correlated failure.
- Require semantic synchronization and verification of understanding.
- Treat imported knowledge as evidence, not unquestionable truth.
- Support local acceptance, rejection, quarantine, reinterpretation, and rollback.

The architecture must remain understandable to engineers as it becomes more capable. Formal precision is useful only when it clarifies behavior, contracts, evidence, or risk.

- Canonical explanations appear once.
- Generated artifacts remain traceable to canonical sources.
- Diagrams expose data flow and state effects.
- Deferred mechanisms are visibly marked as future work.
- Open questions remain open until evidence resolves them.
- Release documentation distinguishes implemented behavior from architectural intention.

- Can importance be represented by one value, a vector, or a structured state?
- How should immediate relevance, long-term utility, novelty, confidence, recurrence, and causal significance interact?
- How quickly should importance change after goals or environments change?
- How can the system prevent manipulation of importance signals?
- Which importance evidence belongs to Semantic Representations, associations, episodes, or connection state?

- What triggers Sleep: elapsed time, accumulated load, instability, resource availability, or explicit policy?
- Which operations are safe during partial online activity?
- How can consolidation distinguish productive abstraction from destructive information loss?
- How should contradictions be preserved, reconciled, or separated?
- What rollback granularity is required after false consolidation?
- Can the same Transformer act safely in online reasoning and offline maintenance?

- What should be replayed, how often, and under which context?
- How can replay preserve diversity rather than repeatedly strengthen dominant memories?
- When should replay weaken or reinterpret knowledge instead of reinforcing it?
- How can corrupted or adversarial memories be prevented from amplification?
- Does replay require explicit episodic storage, or can it operate over semantic structures alone?
- How should replay interact with current goals and Global Evaluation?

- When is weakening sufficient and when is deletion justified?
- How can the system measure the harm caused by stale associations?
- What evidence should authorize creation or removal of structural links?
- How can topology changes remain explainable and reversible?
- How should capacity pressure affect pruning without erasing rare critical knowledge?
- What is the correct boundary between functional and structural learning?

- What state and goals may Global Evaluation inspect?
- How conservative must its learning process be?
- How can objective drift be detected before behavior changes become irreversible?
- Should evaluation produce a scalar, ordered alternatives, constraints, or a structured judgment?
- How does final evaluation interact with planning, action feedback, and memory importance?
- Who or what is authorized to modify the evaluator?

- Which semantic objects are safe and useful to exchange?
- How can two systems verify semantic alignment after message reconstruction?
- How should trust depend on source history, domain competence, confidence, and lineage?
- How can populations preserve useful diversity while sharing discoveries?
- What mechanisms prevent rapid propagation of false semantic representations?
- When should a system request clarification rather than accept an UPDATE candidate?

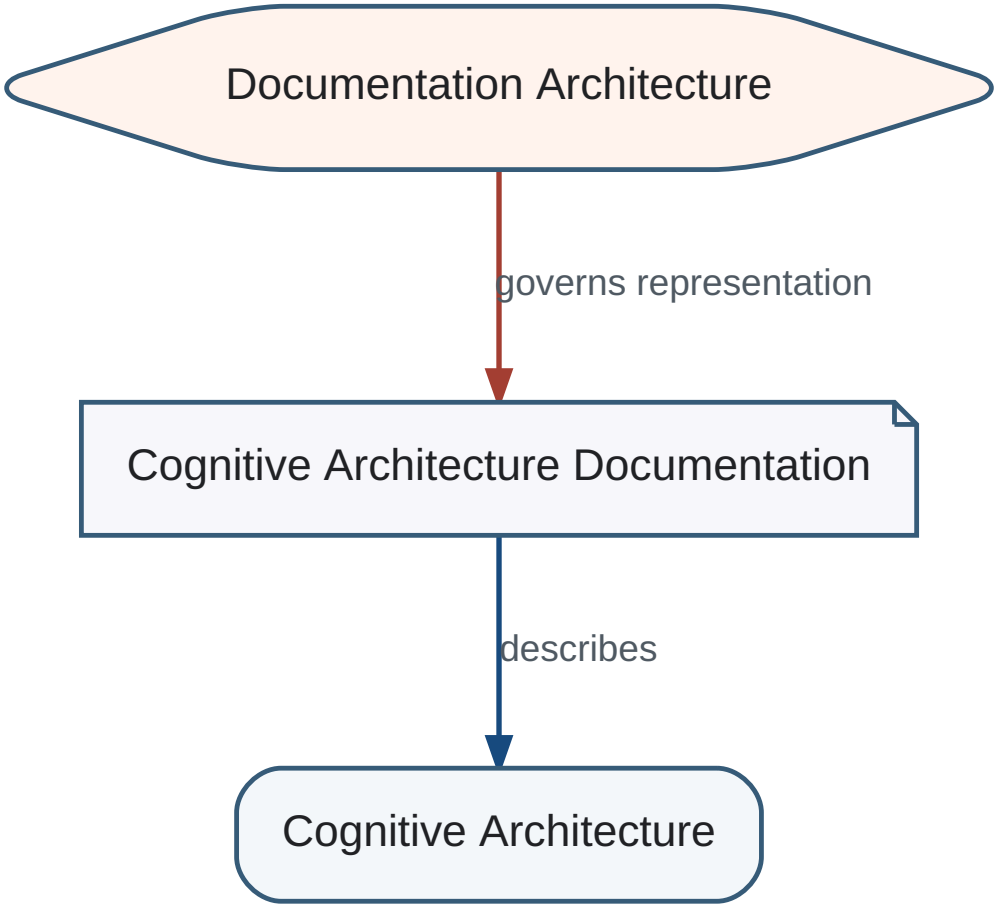
Open questions move into canonical architecture only after a documented decision identifies evidence, alternatives, contracts, invariants, and migration impact.

- A mechanism has a named architectural owner.
- Inputs, outputs, and state effects are explicit.
- Interaction with READ, UPDATE, and Semantic Feedback Learning is defined.
- Failure modes and rollback are documented.
- Implementation status is clearly separated from design intention.
- Tests and generated documentation enforce the accepted decision.

> ### Revision rule
>
> The roadmap is authoritative for direction but not immutable. New evidence may reorder or remove work items, provided the rationale and impact on existing invariants are recorded.

Documentation Architecture governs how knowledge about the Cognitive architecture is represented, related, generated, and validated. It is part of the project Single Source of Truth.

Three-Level Self-Describing Project



Documentation Architecture describes the documentation that describes the Cognitive architecture.

> **DA-0001 — Single Source of Truth**
>
> Canonical knowledge exists once. HTML, PDF, indexes, registries, release notes, and references are generated from canonical sources.

DA-0002 — Separation of Normative and Contextual Content

Architectural invariants and contracts remain distinct from hypotheses, explanatory notes, implementation guidance, and historical analogies.

Typed knowledge objects make epistemic and normative status explicit.

Identifier	Type	Purpose
RN	Research Note	Hypotheses and open research directions
AN	Architectural Note	Rationale and interpretation of architectural decisions
IN	Implementation Note	Reference engineering approaches and non-normative technology choices
HP	Historical Perspective	Historical analogies and motivating examples without claims of historical proof

DA-0003 — Typed Knowledge Objects

Every contextual knowledge object has a declared type, stable identifier, title, status, canonical location, and linkable HTML anchor.

Normative boundary

RN, IN, and HP objects do not change architectural contracts unless a separate canonical architectural revision explicitly adopts their conclusions.

> **DA-0004 — Stable Identifiers**
>
> Identifiers such as RN-0002 remain stable across releases even if wording, location, or generated presentation changes.

> **DA-0005 — Traceability**
>
> Readers must be able to navigate from an architectural decision to its rationale, research motivation, implementation guidance, diagrams, and reverse references.

```
Architecture → Architectural Note → Research Motivation → Implementat
```

The canonical source is progressively evolving toward an explicit knowledge graph. Sections, notes, concepts, diagrams, and references are treated as addressable objects rather than anonymous page fragments.

DA-0006 — Automatic Generation

The generator creates navigation, HTML anchors, typed registries, A–Z entries, cross-references, and future PDF output from canonical data.

- Every typed object must have a unique identifier.
- Every internal reference must resolve.
- Generated HTML must expose a stable anchor for each typed object.
- Registries list canonical objects without duplicating their full text.
- Human readability has priority over excessive formalization.
- Release validation must detect duplicate identifiers and broken references.

Future releases may represent the canonical documentation as an explicit graph of objects and relations while preserving the current human-readable chapter structure.

This registry points to canonical Research Notes. Full text remains at the contextual source location.

- RN-0001 — The Babel Hypothesis
- RN-0002 — Translation as Cognitive State Reconstruction

The Architectural Notes registry is established in cognitive-0.3.20. Canonical AN objects will be added as future material is explicitly classified; no existing normative text is silently reclassified.

The Implementation Notes registry is established in cognitive-0.3.20. Canonical IN objects will be added as future material is explicitly classified; no existing normative text is silently reclassified.

The Historical Perspectives registry is established in cognitive-0.3.20. Canonical HP objects will be added as future material is explicitly classified; no existing normative text is silently reclassified.